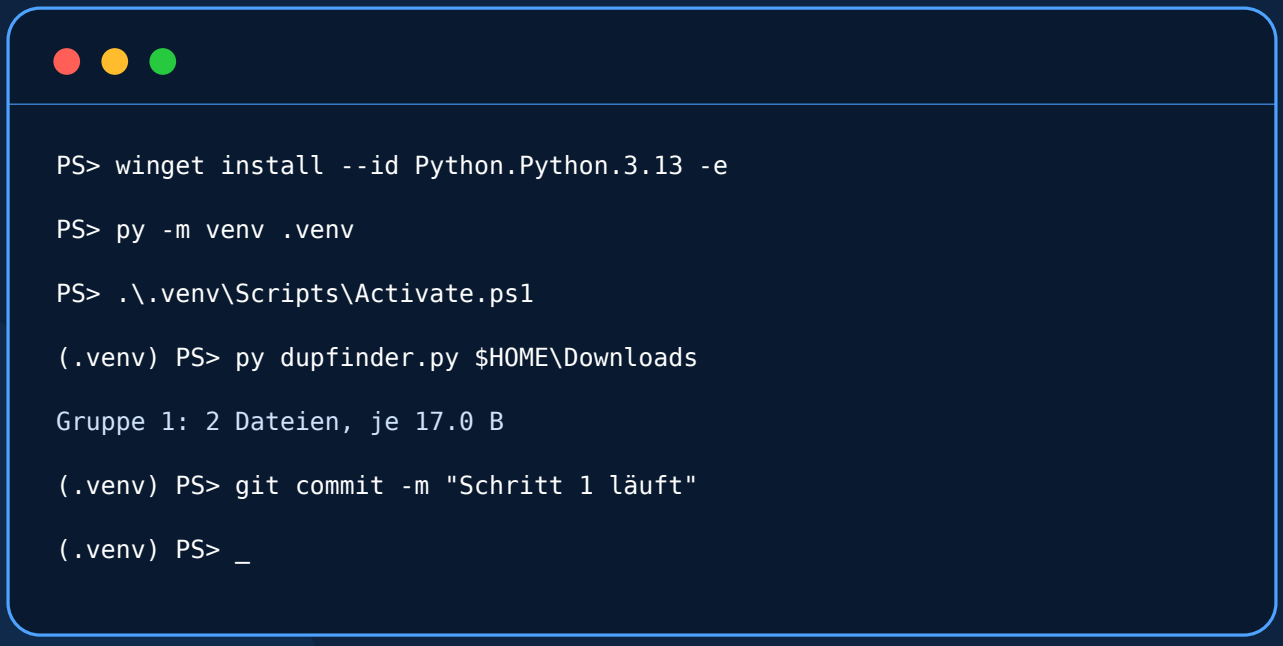
PRAXISHANDBUCH

Vibe Coding unter Windows

Das Praxishandbuch für Einsteiger mit Windows und PowerShell



KLAR BESCHREIBEN. KLEIN BAUEN. SICHER PRÜFEN.

Mit Übungen, Checklisten, Glossar und Index · Ausgabe Oktober 2026

Vibe Coding unter Windows

Das Praxishandbuch für Einsteiger mit Windows-Grundkenntnissen

Von der Idee zum funktionierenden Programm – mit KI-Assistenten, PowerShell und Git.
Mit drei Praxisprojekten, Übungen, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Einleitung: So arbeitest du mit diesem Handbuch 6

Für wen ist dieses Buch? 6

Aufbau 6

Jedes Kapitel funktioniert gleich 6

Konventionen für Code und Terminal 7

Kapitel 1: Was ist Vibe Coding? 8

1.1 Definition und Ursprung 8

1.2 Klassisches Programmieren und Vibe Coding im Vergleich 8

1.3 Was ein KI-Assistent eigentlich tut 9

1.4 Was geht – und was nicht 10

1.5 Dein Mindset: Auftraggeber, Tester, Lerner 10

Übungen 11

Zusammenfassung 11

Kapitel 2: Setup der Umgebung unter Windows 13

2.1 Die Grundausstattung 13

2.2 Der Arbeitsordner 15

2.3 Python und virtuelle Umgebungen 15

2.4 Der Editor 17

2.5 KI-Assistenten einrichten 18

2.6 API-Keys – was sie sind und wie du sie ablegst 19

2.7 Git – dein Sicherheitsnetz 20

2.8 Projektvorlage zum Wiederverwenden 22

Übungen 23

Zusammenfassung 24

Kapitel 3: Der Vibe-Coding-Workflow 25

3.1 Der Grundkreislauf 25

3.2 Vor dem ersten Prompt: Plan und Projektnotiz 25

3.3 Code erzeugen, lesen, ausführen 26

3.4 Iterieren: Änderungswünsche formulieren 28

3.5 Debugging mit KI 28

3.6 Kontext-Management 30

3.7 Wenn nichts mehr geht: Notausgänge . . . 31
3.8 Das Lerntagebuch . . . 31
Übungen . . . 31
Zusammenfassung . . . 32

Kapitel 4: Prompting-Grundlagen für Code . . . 33

4.1 Was einen guten Prompt ausmacht . . . 33
4.2 Bewährte Muster . . . 34
4.3 Kontext liefern: Was, wie viel, in welcher Form . . . 34
4.4 Fehlermeldungen richtig einspeisen . . . 35
4.5 Verständnisfragen – so lernst du nebenbei . . . 36
4.6 Code-Reviews durch KI . . . 36
4.7 Tests als Prompt-Hilfe . . . 37
Übungen . . . 37
Zusammenfassung . . . 38

Kapitel 5: Drei Praxisprojekte . . . 39

Projekt 1: dupfinder – ein CLI-Tool in Python . . . 39
Projekt 2: Ein kleines Web-Projekt . . . 44
Projekt 3: Ein Automatisierungsskript . . . 48
Übungen . . . 53
Zusammenfassung . . . 53

Kapitel 6: Sicherheit und Qualität . . . 54

6.1 Warum Sicherheit hier besonders wichtig ist . . . 54
6.2 API-Keys und Geheimnisse . . . 54
6.3 Gefährliche Befehle erkennen . . . 55
6.4 Typische Schwachstellen in KI-Code . . . 57
6.5 Qualitätssicherung in der Praxis . . . 58
6.6 Abhängigkeiten und Lieferkette . . . 59
6.7 Lizenzfragen . . . 60
6.8 Datenschutz . . . 60
6.9 Wann du KI-Code NICHT vertrauen darfst . . . 61
Übungen . . . 62
Zusammenfassung . . . 62

Kapitel 7: Typische Fallstricke und Troubleshooting . . . 64

7.1 Halluzinationen . . . 64
7.2 Veralteter Code und Versionskonflikte . . . 65

7.3 Abhängigkeitskonflikte . . . 65
7.4 „Es funktioniert, aber ich verstehe es nicht“ . . . 66
7.5 Weitere Fallstricke . . . 67
7.6 Ein Fahrplan für jedes Problem . . . 68
Übungen . . . 69
Zusammenfassung . . . 69

Kapitel 8: Nächste Schritte . . . 70

8.1 Wo stehst du jetzt? . . . 70
8.2 Der Lernpfad in vier Stufen . . . 70
8.3 Die KI als Lehrer einsetzen . . . 71
8.4 Folgeprojekte nach Interesse . . . 71
8.5 Ressourcen zum Weiterlernen . . . 72
8.6 Berufliche Perspektiven realistisch einschätzen . . . 73
8.7 Deine Alltags-Checkliste . . . 73
Schlusswort . . . 74
Übungen . . . 74
Zusammenfassung . . . 74

Glossar . . . 76

Verzeichnis der Codebeispiele . . . 78

Index . . . 80

Vorwort

Du kennst die Kommandozeile unter Windows. Du weißt, wie du in PowerShell oder der Eingabeaufforderung durch Ordner navigierst, Dateien anlegst, verschiebst und suchst, und du hast schon einmal ein Programm per `winget` installiert oder dich über eine rote Fehlermeldung geärgert. Was dir bisher fehlt, ist der Schritt vom **Benutzer** zum **Macher**: Du möchtest nicht nur Programme verwenden, sondern eigene bauen – ein kleines Werkzeug, das dir jeden Tag Arbeit abnimmt, eine Webseite, ein Skript, das Downloads sortiert.

Früher bedeutete das: monatelang eine Programmiersprache lernen. Heute gibt es einen zweiten Weg. KI-Assistenten schreiben Code, erklären ihn, suchen Fehler und schlagen Verbesserungen vor. Dieser Weg hat einen Namen bekommen: **Vibe Coding**. Du beschreibst, was du möchtest; die KI liefert einen Entwurf; du testest, korrigierst die Richtung und wiederholst das, bis das Ergebnis passt.

Dieses Handbuch ist ehrlich. Es verspricht dir nicht, dass du ohne jedes Verständnis professionelle Software bauen kannst. Es zeigt dir aber, wie du mit Windows, PowerShell, einem klaren Kopf und einem guten Arbeitsablauf erstaunlich weit kommst – und wie du dabei sicher und sauber arbeitest. Du lernst nebenbei die wichtigsten Grundlagen des Programmierens, nicht als trockene Theorie, sondern genau dann, wenn du sie brauchst.

> **Gut zu wissen**
>
> **Stand der Inhalte:** Dieses Handbuch ist im Oktober 2026 entstanden. Werkzeuge, Preise, Modellnamen und Menüs ändern sich im KI-Bereich schnell, und auch Windows verschiebt Einstellungen von Version zu Version. Die **Prinzipien** (Kapitel 3, 4, 6, 7) bleiben stabil; für Installationsschritte und Funktionen prüfst du bitte immer die offizielle Dokumentation des jeweiligen Werkzeugs. Die Befehle wurden für Windows 10 und 11 mit PowerShell geschrieben.

Einleitung: So arbeitest du mit diesem Handbuch

Für wen ist dieses Buch?

Du bist hier richtig, wenn du:

- unter Windows im Terminal (PowerShell oder Eingabeaufforderung) sicher genug bist, um Dateien anzulegen, zu verschieben und zu suchen,
- bisher wenig oder gar nicht programmiert hast,
- mit KI-Assistenten (zum Beispiel Claude, ChatGPT oder GitHub Copilot) eigene kleine Software bauen möchtest.

Du brauchst **kein** Vorwissen in Python, JavaScript oder Git. Das Wichtigste dazu lernst du in den Kapiteln.

Aufbau

Das Handbuch hat acht Kapitel, die aufeinander aufbauen:

Kapitel	Thema	Du kannst danach
1	Was ist Vibe Coding?	Erwartungen und Grenzen realistisch einschätzen
2	Setup unter Windows	Terminal, Editor, KI-Assistent, Git und Projektordner einrichten
3	Der Workflow	in kleinen Schritten vom Prompt zum laufenden Code kommen
4	Prompting für Code	präzise Anweisungen und Fehlermeldungen formulieren
5	Drei Praxisprojekte	CLI-Tool, Web-Projekt und Automatisierungsskript bauen
6	Sicherheit und Qualität	Keys, Daten und Code verantwortungsvoll behandeln
7	Fallstricke	typische Probleme erkennen und beheben
8	Nächste Schritte	deinen Lernpfad planen

Am Ende findest du ein **Glossar**, ein **Verzeichnis der Codebeispiele** und einen **Index**.

Jedes Kapitel funktioniert gleich

Jedes Kapitel beginnt mit **Lernzielen** und endet mit **Übungen** und einer **Zusammenfassung**. Dazwischen findest du Erklärungen und Codebeispiele. Die Beispiele sind nummeriert (zum Beispiel „Beispiel 3.2“), damit du sie im Verzeichnis am Ende wiederfindest.

Es gibt farbige Boxen:

> **WARNUNG – Sicherheitsrisiko**

Rote Boxen warnen vor echten Risiken: Datenverlust, ausgespähte Zugangsdaten, gefährliche Befehle. Lies sie immer.

TIPP – Profi-Trick

Grüne Boxen enthalten Tricks, die den Alltag deutlich leichter machen.

Gut zu wissen

Violette Boxen geben Hintergrundwissen, das du nicht sofort brauchst, das aber Zusammenhänge klärt.

Konventionen für Code und Terminal

Codeblöcke mit `PS>` am Zeilenanfang sind PowerShell-Befehle. Das `PS>` tippst du **nicht** mit; es steht für die Eingabeaufforderung. Zeilen ohne `PS>` darunter sind die Ausgabe. Ein `#` in einer PowerShell-Zeile leitet einen Kommentar ein.

Als Arbeitsordner verwenden wir durchgehend `$HOME\vibe`. In PowerShell steht `$HOME` für dein Benutzerverzeichnis, also `C:\Users\DEINNAME`. Pfade schreiben wir mit Backslash (`\`), PowerShell versteht aber auch den normalen Schrägstrich.

Gut zu wissen

Windows-Version oder Linux-Version? Es gibt dieses Handbuch auch für Linux. Der Inhalt ist gleich, nur die Werkzeuge und Befehle unterscheiden sich. Wenn du auf Windows arbeitest, aber lieber die Linux-Umgebung nutzen möchtest, kannst du sie mit WSL (Windows-Subsystem für Linux) direkt in Windows starten. Mehr dazu in Kapitel 2.

TIPP – Profi-Trick

Arbeite jedes Beispiel wirklich am Rechner durch. Beim Programmieren lernst du fast nur durch Tun. Wer nur liest, versteht später zwar die Wörter, aber nicht die Probleme.

Kapitel 1: Was ist Vibe Coding?

Lernziele

- Du kannst Vibe Coding definieren und vom klassischen Programmieren abgrenzen.
- Du kennst den Ursprung des Begriffs und die Bandbreite seiner Bedeutung.
- Du verstehst, was ein KI-Assistent technisch tut – und warum er Fehler macht.
- Du kannst realistisch einschätzen, was mit Vibe Coding geht und was nicht.

1.1 Definition und Ursprung

Der Begriff **Vibe Coding** wurde im Februar 2025 durch den KI-Forscher Andrej Karpathy populär. In einem Beitrag beschrieb er eine Arbeitsweise, bei der man sich „den Vibes hingibt“: Man sagt der KI in normaler Sprache, was man möchte, übernimmt ihre Vorschläge, ohne jede Zeile im Detail zu prüfen, und korrigiert nur, wenn etwas nicht funktioniert. Der Begriff verbreitete sich rasend schnell und wurde 2025 sogar von einem großen Wörterbuchverlag zum Wort des Jahres gekürt.

Karpathys ursprüngliche Beschreibung meinte eine bewusst lockere Arbeitsweise für Wochenendprojekte und Experimente. Im Alltag hat der Begriff sich verbreitert. Heute fallen darunter drei verschiedene Dinge:

Variante	Beschreibung	Typischer Einsatz
Reines Vibe Coding	Code wird nicht gelesen, nur das Ergebnis getestet	Wegwerf-Prototypen, Spielereien
KI-gestütztes Programmieren	Die KI schreibt, der Mensch liest und versteht den Code	professionelle Entwicklung
Geführtes Lernen mit KI	Die KI baut und erklärt, der Mensch lernt dabei	genau dein Weg mit diesem Buch

In diesem Handbuch gehen wir bewusst den **mittleren bis dritten Weg**: Du lässt dir Code schreiben, aber du lernst, ihn zu testen, in groben Zügen zu lesen und zu hinterfragen. Das ist langsamer als reines Vibe Coding, aber sehr viel robuster.

Gut zu wissen

Faustregel: Je mehr Schaden ein Fehler anrichten kann, desto weniger darfst du „nach Gefühl“ arbeiten. Ein Skript, das Fotos umbenennt, darf mal schiefgehen (mit Backup). Ein Skript, das Passwörter verwaltet oder Geld bewegt, darfst du nie ungeprüft einsetzen.

1.2 Klassisches Programmieren und Vibe Coding im Vergleich

Beim klassischen Programmieren schreibst du jede Anweisung selbst. Du musst die Syntax einer Sprache beherrschen, die Standardbibliothek kennen und Fehler selbst suchen. Dafür weißt du genau, was dein Programm tut.

Beim Vibe Coding verschiebt sich deine Rolle: Du wirst vom Schreibenden zum **Auftraggeber, Tester und Prüfer**. Die Fähigkeiten, die du brauchst, ändern sich:

Fähigkeit	Klassisch	Vibe Coding
Syntax auswendig	sehr wichtig	kaum nötig
Problem klar beschreiben	wichtig	**entscheidend**
Ergebnis testen	wichtig	**entscheidend**
Fehlermeldungen lesen	wichtig	wichtig
Aufgabe in kleine Schritte teilen	wichtig	**entscheidend**
Code im Detail verstehen	Pflicht	wünschenswert, wächst mit der Zeit
Versionskontrolle (Git)	wichtig	**sehr wichtig** (Sicherheitsnetz)

Beachte: Die letzte Zeile ist kein Zufall. Wenn eine KI in Sekunden hunderte Zeilen ändert, brauchst du ein Sicherheitsnetz, mit dem du jederzeit zum letzten funktionierenden Stand zurückkehren kannst. Dieses Netz heißt Git, und du richtest es in Kapitel 2 ein.

1.3 Was ein KI-Assistent eigentlich tut

Ein **Sprachmodell** (englisch: Large Language Model, LLM) wurde mit riesigen Mengen Text und Code trainiert. Es hat dabei Muster gelernt: Wie sieht eine Python-Funktion aus, die eine Datei liest? Wie sieht eine typische Fehlermeldung aus, und was ist meist die Ursache?

Wenn du eine Anfrage stellst, **berechnet** das Modell Wort für Wort (genauer: Token für Token), was wahrscheinlich als Nächstes kommt. Es „führt dabei keinen Code aus“ und „schlägt nichts nach“, solange es nicht dafür ausgerüstet wurde. Daraus folgen drei wichtige Eigenschaften:

- **Es klingt immer selbstsicher.** Auch falsche Antworten sind flüssig formuliert.
- **Es kann Dinge erfinden** (Halluzinationen): Funktionen, die es nicht gibt, Kommandozeilenoptionen, die nie existiert haben, Bibliotheken mit erfundenem Namen.
- **Sein Wissen hat ein Datum.** Neuere Versionen von Bibliotheken kennt es möglicherweise nicht.

Moderne Assistenten haben diese Schwächen durch Werkzeuge abgemildert: Manche können Dateien in deinem Projekt lesen, Befehle im Terminal ausführen, Tests laufen lassen und im Web nachschlagen. Das macht sie deutlich nützlicher, aber nicht unfehlbar.

Drei Arten von Assistenten

1. **Chat-Assistenten im Browser** (zum Beispiel Claude, ChatGPT, Gemini): Du kopierst Code hin und her. Ideal zum Einstieg und für Erklärungen.
2. **Editor-Assistenten** (zum Beispiel GitHub Copilot, Continue oder KI-Editoren wie Cursor): Die KI sitzt im Editor, sieht deine geöffneten Dateien und schlägt Änderungen direkt im Code vor.
3. **Terminal-Agenten** (zum Beispiel Claude Code, Aider, Codex CLI oder Gemini CLI): Die KI läuft in deinem Terminal, liest dein Projektverzeichnis, ändert Dateien und führt Befehle aus – mit deiner Erlaubnis.

> **Gut zu wissen**
>
> Für Windows-Nutzer sind Terminal-Agenten besonders spannend, weil sie genau in der PowerShell arbeiten, die du ohnehin kennst. Sie sind aber auch die Werkzeuge mit dem größten Schadenspotenzial, weil sie Befehle ausführen. Mehr dazu in Kapitel 6.

1.4 Was geht – und was nicht

Das gelingt in der Regel gut

- **Kleine bis mittlere Werkzeuge:** Skripte für Dateien, Texte, Backups, Umbenennungen, Auswertungen.
- **Prototypen und Webseiten:** Eine einfache Seite, ein Formular, ein kleiner Rechner, ein Dashboard.
- **Glue-Code:** Ein Programm mit einem anderen verbinden, Daten von A nach B bringen.
- **Gängige Muster:** Alles, was tausendfach im Netz vorkommt (Login-Formular, CSV einlesen, Tabelle sortieren).
- **Lernen und Erklären:** „Erkläre mir diese Zeile“ ist eine der besten Anwendungen überhaupt.

Hier wird es schwierig

- **Große, gewachsene Systeme**, in denen die KI den Gesamtzusammenhang nicht überblickt.
- **Sicherheitskritische Bereiche:** Authentifizierung, Kryptografie, Zahlungsverkehr.
- **Sehr neue oder seltene Technik**, über die es wenig Material gibt.
- **Unklare Anforderungen:** Wenn du selbst nicht weißt, was du möchtest, kann die KI es nicht erraten.
- **Performance und Feinheiten:** Code, der „läuft“, ist nicht automatisch schnell oder wartbar.

Die ehrliche Erwartungskurve

Viele Einsteiger erleben denselben Verlauf: Die ersten Erfolge sind verblüffend – nach zehn Minuten läuft ein Programm. Dann wächst das Projekt, die KI ändert Dinge, die sie nicht ändern sollte, und plötzlich geht nichts mehr. Das ist normal. Der Rest dieses Buches zeigt dir, wie du diese zweite Phase überstehst: mit kleinen Schritten, Git, Tests und gezielten Fragen.

> **WARNUNG – Sicherheitsrisiko**
>
> **Gefährliche Illusion:** „Es läuft, also ist es richtig.“ Code kann ohne Fehlermeldung laufen und trotzdem das Falsche tun – zum Beispiel Dateien überschreiben, Daten verlieren oder Lücken öffnen. Teste immer an Kopien und nie zuerst an deinen echten Daten.

1.5 Dein Mindset: Auftraggeber, Tester, Lerner

Drei Rollen begleiten dich durch das ganze Handbuch:

1. **Auftraggeber:** Du formulierst klar, was gebaut werden soll und was nicht.
2. **Tester:** Du führst das Ergebnis aus, probierst Randfälle und prüfst, ob es dein Ziel erfüllt.
3. **Lerner:** Du fragst bei jedem unverständlichen Teil nach. Aus „Es funktioniert, aber ich verstehe es nicht“ wird mit der Zeit „Ich verstehe die Hauptidee“.

Ein sinnvolles Ziel ist nicht, jede Zeile zu verstehen, sondern **den Aufbau**: Welche Teile gibt es? Welcher Teil tut was? Wo würde ich suchen, wenn etwas kaputt ist?

Dein erster Mini-Test: Hallo Welt mit einer KI

Bevor wir die Umgebung einrichten, ein kurzer Vorgeschmack. Öffne einen Chat-Assistenten im Browser und gib diese Anfrage ein:

Beispiel 1.1: Eine erste Anfrage an die KI

```
Schreibe mir ein kleines Python-Programm für Windows. Es soll mich nach meinem
Namen fragen und dann "Hallo, <Name>!" ausgeben. Erkläre danach jede Zeile
in einem Satz, als wäre ich Anfänger.
```

Du erhältst etwa ein Programm wie dieses:

Beispiel 1.2: Typische Antwort – ein Hallo-Programm

```
name = input("Wie heißt du? ")
print(f"Hallo, {name}!")
```

Speichere es als `$HOME\hallo.py` und starte es im Terminal:

Beispiel 1.3: Programm starten

```
PS> py $HOME\hallo.py
Wie heißt du? Eugen
Hallo, Eugen!
```

Was ist passiert? `input(...)` zeigt eine Frage an und wartet auf deine Eingabe. Das Ergebnis speichert das Programm in der **Variablen** `name`. `print(...)` gibt Text aus; das `f` vor den Anführungszeichen erlaubt, mit `{name}` den Inhalt der Variable in den Text einzusetzen. Mehr ist das erste Programm nicht – und du hast es gerade durch eine Beschreibung in normaler Sprache entstehen lassen.

Übungen

Übung

1. Formuliere in eigenen Worten den Unterschied zwischen reinem Vibe Coding und KI-gestütztem Programmieren.
2. Nenne je ein Beispielprojekt, bei dem „nach Gefühl“ arbeiten okay ist, und eines, bei dem es riskant wäre.
3. Lass dir von einem Chat-Assistenten das Hallo-Programm aus Beispiel 1.1 schreiben. Bitte ihn dann, es so zu erweitern, dass es dich zusätzlich nach deinem Alter fragt und ausrechnet, wie alt du in zehn Jahren bist. Erkläre danach jede neue Zeile.
4. Frage die KI absichtlich nach etwas, das es nicht gibt (zum Beispiel „Zeig mir die Python-Funktion `datei.zauberhaft_lesen()`“). Wie reagiert sie?

Lösungshinweis

Zu 4: Gute Assistenten sagen, dass die Funktion nicht existiert, und bieten eine Alternative an. Manchmal erfinden sie jedoch eine Erklärung. Genau deshalb testest du Code immer aus. Wenn du im Terminal `py -c "help(open)"` aufrufst, siehst du die echte Dokumentation.

Zusammenfassung

Zusammenfassung

- Vibe Coding bedeutet, Software durch Beschreibung in natürlicher Sprache mit einer KI zu bauen und iterativ zu verbessern.

- Der Begriff wurde 2025 durch Andrej Karpathy populär; das Spektrum reicht von „Code nicht lesen“ bis zu sorgfältig geprüfter KI-Unterstützung.
- Deine Rolle: Auftraggeber, Tester und Lerner – nicht Tipper.
- KI-Modelle berechnen wahrscheinliche Fortsetzungen: Sie klingen sicher, können aber halluzinieren und kennen neuere Entwicklungen eventuell nicht.
- Es gibt drei Arten von Assistenten: Chat, Editor-Integration und Terminal-Agenten.
- Je höher der mögliche Schaden, desto weniger darfst du ungeprüft vertrauen.

Kapitel 2: Setup der Umgebung unter Windows

Lernziele

- Du richtest Windows Terminal, PowerShell, Python und Git ein.
- Du installierst einen Editor (VS Code oder Neovim) und kennst die Alternativen.
- Du wählst einen KI-Assistenten und weißt, wie ein API-Key funktioniert.
- Du legst dein erstes Git-Repository an und machst deinen ersten Commit.

2.1 Die Grundausstattung

Eine gute Vibe-Coding-Umgebung besteht aus fünf Bausteinen:

1. **Terminal** – Windows Terminal mit PowerShell.
2. **Programmiersprache** – wir nutzen Python und später etwas HTML/JavaScript.
3. **Editor** – zum Lesen und Anpassen von Code.
4. **KI-Assistent** – dein Co-Autor.
5. **Git** – dein Sicherheitsnetz.

Terminal: Windows Terminal und PowerShell

Unter Windows 11 ist **Windows Terminal** vorinstalliert, unter Windows 10 bekommst du es kostenlos aus dem Microsoft Store. Es öffnet standardmäßig PowerShell. Auf deinem Rechner ist wahrscheinlich die ältere **Windows PowerShell 5.1** vorhanden; die moderne **PowerShell 7** ist ein eigenes Programm, das du parallel installieren kannst. Für dieses Handbuch reicht beides, PowerShell 7 verhält sich in einigen Punkten angenehmer (siehe Kapitel 7).

Systemcheck

Öffne Windows Terminal und prüfe, was vorhanden ist:

Beispiel 2.1: Vorhandene Werkzeuge prüfen

```
PS> $PSVersionTable.PSVersion
Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      22621  4391
PS> winget --version
v1.9.25200
PS> Get-Command git, py, code -ErrorAction SilentlyContinue
```

Fehlt ein Programm, taucht es in der Ausgabe von `Get-Command` nicht auf. Das ist normal; wir installieren es gleich.

Pakete installieren mit winget

winget ist der Paketmanager von Windows, vergleichbar mit `apt` oder `dnf` unter Linux. Er ist unter Windows 11 und aktuellen Windows-10-Versionen vorhanden.

Beispiel 2.2: Grundpakete mit winget installieren

```
PS> winget search Python.Python
PS> winget install --id Python.Python.3.13 -e
PS> winget install --id Git.Git -e
PS> winget install --id Microsoft.VisualStudioCode -e
PS> winget install --id Microsoft.PowerShell -e
```

Mit `winget search` prüfst du vorher die genauen Paketnamen und wählst eine aktuelle Python-3-Version; die Versionsnummer im Beispiel kann inzwischen überholt sein. Die Option `-e` bedeutet „genau diese ID“. **Schließe danach das Terminal und öffne es neu**, damit Windows die neuen Programme im Suchpfad (`PATH`) findet.

Nützliche Zusatzwerkzeuge:

Beispiel 2.3: Zusatzwerkzeuge installieren

```
PS> winget install --id jqlang.jq -e
PS> winget install --id BurntSushi.ripgrep.MSVC -e
PS> winget install --id sharkdp.fd -e
PS> winget install --id Neovim.Neovim -e
```

Wofür sind die Werkzeuge gut?

Werkzeug	Zweck
`git`	Versionskontrolle, Sicherheitsnetz
`py` (Python-Launcher)	startet Python; legt virtuelle Umgebungen an
`jq`	JSON lesen und filtern
`tree`	Ordnerstruktur anzeigen (in Windows eingebaut: `tree /F`)
`ripgrep` (`rg`)	sehr schnelle Textsuche in Projekten
`fd`	schnelle Dateisuche
`Select-String`	PowerShell-Befehl zum Durchsuchen von Text, Gegenstück zu `grep`
`Get-Help`	eingebaute Hilfe zu jedem PowerShell-Befehl

Wenn du `grep`, `ls` und `cd` kennst, helfen dir diese Entsprechungen:

Aus der Linux-Welt	PowerShell
`ls`	`Get-ChildItem` (Alias `ls`, `dir`)
`cd`	`Set-Location` (Alias `cd`)
`cat`	`Get-Content` (Alias `cat`)
`grep`	`Select-String`
`cp`, `mv`, `rm`	`Copy-Item`, `Move-Item`, `Remove-Item`
`mkdir -p`	`New-Item -ItemType Directory` (oder `mkdir`)
`which`	`Get-Command`
`chmod`	`icacls` (Zugriffsrechte unter Windows)

> **Gut zu wissen**

Python startet den Microsoft Store? Wenn `python` im Terminal den Microsoft Store öffnet, ist ein „App-Ausführungsalias“ aktiv. Starte Python stattdessen mit `py` (das funktioniert nach der Installation immer), oder schalte unter *Einstellungen → Apps → Erweiterte App-Einstellungen → App-Ausführungsaliase* die Python-Aliase aus.

TIPP – Profi-Trick

Du möchtest unter Windows lieber eine echte Linux-Umgebung? Mit `wsl --install` (in einem Administrator-Terminal) richtet Windows ein Linux-Subsystem ein, meist mit Ubuntu. Dort kannst du alles aus der Linux-Version dieses Handbuchs verwenden. Für den Einstieg ist das nicht nötig; Python, Git und VS Code laufen unter Windows hervorragend.

2.2 Der Arbeitsordner

Ordnung im Dateisystem spart später Nerven. Wir legen einen festen Platz für alle Projekte an:

Beispiel 2.4: Arbeitsordner anlegen

```
PS> "projekte","experimente","notizen" | ForEach-Object {
      New-Item -ItemType Directory -Force "$HOME\vibe\$_" | Out-Null }
PS> Set-Location "$HOME\vibe"
PS> tree /F
C:.
├───experimente
├───notizen
└───projekte
```

- `projekte\` – Dinge, die du behalten und mit Git verwalten willst.
- `experimente\` – Wegwerf-Versuche.
- `notizen\` – dein Lerntagebuch (siehe Kapitel 3).

TIPP – Profi-Trick

Lege den Ordner **nicht** in OneDrive-synchronisierten Ordnern wie *Dokumente* oder *Desktop* an. Die Synchronisierung stört bei vielen kleinen Dateien (zum Beispiel in `.venv` und `.git`) und kann Probleme verursachen. `C:\Users\DEINNAME\vibe` ist meist unproblematisch.

2.3 Python und virtuelle Umgebungen

Python-Programme brauchen oft zusätzliche Bibliotheken. Damit sich Projekte nicht gegenseitig stören, bekommt jedes Projekt seine eigene **virtuelle Umgebung** (englisch: virtual environment, kurz *venv*): einen Ordner mit einer privaten Python-Installation.

Beispiel 2.5: Virtuelle Umgebung anlegen und aktivieren

```
PS> py --version
Python 3.13.0
PS> Set-Location "$HOME\vibe\experimente"
PS> mkdir test; Set-Location test
PS> py -m venv .venv
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> Get-Command python | Select-Object -ExpandProperty Source
C:\Users\DEINNAME\vibe\experimente\test\.venv\Scripts\python.exe
(.venv) PS> pip install requests
(.venv) PS> deactivate
PS>
```

Nach `Activate.ps1` erscheint `(.venv)` vor der Eingabeaufforderung. Alles, was du jetzt mit `pip install` installierst, landet nur in diesem Ordner. Mit `deactivate` verlässt du die Umgebung wieder.

Fehler: „Die Ausführung von Skripts ist auf diesem System deaktiviert“

Windows blockiert PowerShell-Skripte standardmäßig. Beim Aktivieren der venv erscheint dann eine rote Meldung zur **Ausführungsrichtlinie** (englisch: execution policy). Die sichere Lösung für dein Benutzerkonto:

Beispiel 2.6: Ausführungsrichtlinie für dein Konto lockern (einmalig)

```
PS> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
PS> Get-ExecutionPolicy -List
```

`RemoteSigned` erlaubt eigene, lokal erstellte Skripte und verlangt für aus dem Internet geladene Skripte eine digitale Signatur. Das ist der übliche Kompromiss für Entwickler.

> **WARNUNG – Sicherheitsrisiko**
>
> Setze die Ausführungsrichtlinie **nicht** auf `Unrestricted` oder `Bypass` und starte dafür kein Terminal „als Administrator“ nur zur Bequemlichkeit. Die Richtlinie ist ein Sicherheitsnetz gegen versehentlich ausgeführte Fremdskripte. Alternativ kannst du die Aktivierung ganz umgehen und Python direkt aus der venv aufrufen: `.\.venv\Scripts\python.exe skript.py`.

> **WARNUNG – Sicherheitsrisiko**
>
> Installiere Python-Pakete nicht global mit Administratorrechten. Das kann zu Konflikten zwischen Projekten und zu schwer nachvollziehbaren Fehlern führen. Die richtige Antwort ist immer: **eine venv verwenden**.

Die Liste der installierten Pakete speicherst du so, damit andere (und du selbst später) sie neu installieren können:

Beispiel 2.7: Abhängigkeiten festhalten und wiederherstellen

```
(.venv) PS> pip freeze | Set-Content requirements.txt -Encoding utf8
(.venv) PS> Get-Content requirements.txt
certifi==...
requests==...
PS> pip install -r requirements.txt     # später, in einer neuen venv
```

> **WARNUNG – Sicherheitsrisiko**

Schreibe `pip freeze > requirements.txt` in **Windows PowerShell 5.1** nicht mit dem einfachen `>`: Es speichert die Datei dort in UTF-16, und `pip` kann sie dann nicht lesen. Nutze wie oben `Set-Content -Encoding utf8`.

Gut zu wissen

Auch `-Encoding utf8` ist nicht in jeder PowerShell-Version gleich: **Windows PowerShell 5.1** schreibt damit UTF-8 **mit** einer sogenannten BOM (Byte Order Mark, ein paar unsichtbare Kennzeichnungs-Bytes am Dateianfang); **PowerShell 7** schreibt UTF-8 standardmäßig **ohne** BOM. Für `requirements.txt`, `.gitignore` und die meisten Textdateien in diesem Buch ist das unkritisch, weil die lesenden Programme (`pip`, `git`) beide Varianten verstehen. Bei eigenem Code, der Textdateien exakt byteweise vergleicht, oder bei manchen älteren Programmen kann die BOM aber zu Überraschungen führen – prüfe im Zweifel mit `Format-Hex datei.txt -Count 8`, ob die ersten Bytes `EF BB BF` enthalten.

2.4 Der Editor

Ein Editor ist dein Fenster in den Code. Du musst ihn nicht perfekt beherrschen, aber du solltest Code lesen, kleine Änderungen vornehmen und Dateien schnell wechseln können.

Option A: Visual Studio Code (empfohlen für Einsteiger)

VS Code ist grafisch, hat eine integrierte Terminal-Leiste und Erweiterungen für alle gängigen KI-Assistenten. Du hast es in Beispiel 2.2 bereits installiert. Starte es aus dem Projektordner, damit es den Ordner als Arbeitsbereich öffnet:

Beispiel 2.8: Ordner in VS Code öffnen

```
PS> Set-Location "$HOME\vibe\projekte"
PS> code .
```

Gut zu wissen

VS Code von Microsoft enthält Telemetrie und Microsoft-Dienste. Wer das nicht möchte, findet in **VSCodium** einen Build ohne diese Bestandteile; manche Erweiterungen sind dort allerdings nicht oder nur eingeschränkt verfügbar. Für den Einstieg ist beides in Ordnung. Beim Öffnen eines neuen Ordners fragt VS Code, ob du dem Autor vertraust („Arbeitsbereich-Vertrauen“): Bestätige das nur bei Ordnern, die du selbst angelegt hast.

Nützliche Tastenkürzel zum Anfang:

Kürzel	Wirkung
`Strg+P`	Datei per Namen öffnen
`Strg+Umschalt+P`	Befehlspalette (alles ist hier auffindbar)
`Strg+ö`	Integriertes Terminal ein/aus
`Strg+Z`	Rückgängig
`Strg+F` / `Strg+Umschalt+F`	Suchen in Datei / im ganzen Projekt

TIPP – Profi-Trick

VS Code kann das integrierte Terminal direkt mit PowerShell öffnen. Wenn die venv im Projekt liegt, erkennt VS Code sie meist automatisch und bietet sie als Python-Interpreter an (unten rechts oder über die Befehlspalette: *Python: Interpreter auswählen*).

Option B: Neovim

Neovim läuft im Terminal und ist extrem schnell, hat aber eine steile Lernkurve. Wenn du dich fürs Terminal begeisterst, lohnt es sich – für den Anfang aber nicht nötig.

Beispiel 2.9: Neovim starten und das Tutorial öffnen

```
PS> nvim --version | Select-Object -First 1
PS> nvim +Tutor               # interaktives Tutorial, ca. 30 Minuten
```

Die fünf wichtigsten Dinge in Neovim: Mit `i` wechselst du in den **Einfügemodus** (Text tippen), mit `Esc` zurück in den **Normalmodus**. `:w` speichert, `:q` beendet, `:wq` macht beides, `:q!` beendet ohne Speichern.

TIPP – Profi-Trick

Wer nur kurz eine Konfigurationsdatei ändern will, kommt mit `notepad datei.txt` aus. Das ist völlig legitim. Aktuelle Windows-11-Versionen haben außerdem den einfachen Konsoleneditor `edit`.

2.5 KI-Assistenten einrichten

Du hast drei Wege, die auch kombinierbar sind. Beginne mit Weg 1, bis du dich sicher fühlst.

Weg 1: Chat im Browser

Kein Setup nötig: Konto anlegen, Chat öffnen, Code hin- und herkopieren. Vorteil: sehr sicher, weil die KI keinen Zugriff auf dein System hat. Nachteil: viel Kopieren, und die KI sieht nur, was du ihr zeigst.

Weg 2: Erweiterung im Editor

Im VS-Code-Erweiterungsbereich (`Strg+Umschalt+X`) findest du Assistenten wie GitHub Copilot, Continue und weitere. Sie zeigen Vorschläge direkt im Code und können mit der KI über geöffnete Dateien chatten. Anmeldung und Abrechnung laufen je nach Anbieter über ein Konto oder einen eigenen API-Key.

Weg 3: Terminal-Agent

Ein Terminal-Agent läuft in deinem Projektordner, liest Dateien und schlägt Änderungen vor. Beispiele sind Claude Code, Aider, Codex CLI und Gemini CLI. Ob ein Werkzeug unter Windows nativ läuft oder besser innerhalb von WSL, ändert sich von Version zu Version, und auch die Installationsschritte sind im Fluss. Daher gilt: **folge der offiziellen Anleitung.** Bei Claude Code findest du sie in der Dokumentation unter docs.claude.com (Bereich Claude Code).

Der typische Ablauf bei jedem Terminal-Agenten ist gleich:

Beispiel 2.10: Typischer Ablauf mit einem Terminal-Agenten

```
PS> Set-Location "$HOME\vibe\projekte\mein-projekt"   # IMMER erst in den richtigen Ordner
PS> git status                                         # sauberer Stand? Sonst erst committen
PS> <agent-name>                                       # Agent starten (Name je nach Werkzeug)
> Lies bitte die README und erkläre mir, was dieses Projekt tut.
```

> **WARNUNG – Sicherheitsrisiko**
>
> Starte einen Terminal-Agenten **nie** in deinem Benutzerverzeichnis (`C:\Users\DEINNAME`) oder im Laufwerks-Stamm (`C:\`). Er hat dann Zugriff auf alle deine Dateien, auch auf `.ssh`, Browserdaten-Ordner und Passwortexporte. Wechsle immer zuerst in den Projektordner. Erlaube Befehle einzeln, lies sie, bevor du „Ja“ sagst, und starte den Agenten nicht in einem Terminal mit Administratorrechten.

Was kostet das?

Die Preismodelle ändern sich häufig: Manche Dienste bieten kostenlose Kontingente, andere Abos mit Nutzungslimits, wieder andere rechnen pro verarbeitetem Text (Token) ab. Prüfe die aktuellen Preise beim Anbieter und setze nach Möglichkeit ein **Ausgabenlimit** im Konto.

2.6 API-Keys – was sie sind und wie du sie ablegst

Ein **API-Key** ist ein langer Zugangsschlüssel (eine Zeichenkette), mit dem ein Programm bei einem Dienst „angemeldet“ ist. Wer den Key hat, kann den Dienst **auf deine Rechnung** nutzen. Behandle ihn wie ein Passwort.

Der Haken am Terminal-Verlauf

PowerShell merkt sich eingegebene Befehle in einer Verlaufsdatei. Tippst du einen Key direkt in einen Befehl (zum Beispiel `setx KEY "geheim"`), steht er danach im Klartext in dieser Datei. Gib einen Key daher **nie** als Teil eines Befehls ein. Den Pfad der Verlaufsdatei zeigt dir dieser Befehl:

Beispiel 2.11: Verlaufsdatei von PowerShell finden

```
PS> (Get-PSReadLineOption).HistorySavePath
C:\Users\DEINNAME\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
```

Variante A: Key in einer geschützten Datei

Der Key liegt in einer Datei, die nur du lesen darfst, und wird bei Bedarf in die aktuelle Sitzung geladen. Du tippst den Key im **Editor** ein, nicht im Terminal:

Beispiel 2.12: API-Key sicher ablegen

```
PS> New-Item -ItemType Directory -Force "$HOME\.config\vibe" | Out-Null
PS> $datei = "$HOME\.config\vibe\keys.ps1"
PS> New-Item -ItemType File -Force $datei | Out-Null
PS> notepad $datei
# In Notepad eine Zeile eintragen und speichern:
#   $env:ANTHROPIC_API_KEY = "hier-dein-key"
PS> icacls $datei /inheritance:r /grant:r "${env:USERNAME}:(R,W)"
PS> icacls $datei
```

Der `icacls`-Befehl entfernt geerbte Rechte und erlaubt nur deinem Benutzerkonto Lesen und Schreiben; das ist das Windows-Gegenstück zu `chmod 600`. Bei Bedarf lädst du die Datei in die aktuelle Terminal-Sitzung:

Beispiel 2.13: Key in die Sitzung laden und prüfen

```
PS> . "$HOME\.config\vibe\keys.ps1"
PS> if ($env:ANTHROPIC_API_KEY) { "Key gesetzt" } else { "Key fehlt" }
Key gesetzt
```

Der Punkt mit Leerzeichen am Anfang („dot-sourcing“) führt die Datei in der aktuellen Sitzung aus, damit die Variable dort erhalten bleibt. Beachte: Wir geben den Key nie mit `echo $env:ANTHROPIC_API_KEY` aus.

Variante B: Dauerhafte Benutzer-Umgebungsvariable

Wer den Key dauerhaft in jeder Sitzung haben möchte, trägt ihn über die grafische Oberfläche ein: Suche im Startmenü nach **„Umgebungsvariablen für dieses Konto bearbeiten“**, klicke unter „Benutzervariablen“ auf *Neu* und gib Name und Wert ein. Neue Terminals sehen die Variable danach. Auch so landet nichts im Terminal-Verlauf.

> **WARNUNG – Sicherheitsrisiko**
>
> **Niemals** einen API-Key direkt in eine Programmdatei schreiben, in einen Chat einfügen oder in ein Git-Repository committen. Bots durchsuchen öffentliche Repositories ständig nach Keys; ein veröffentlichter Key ist oft binnen Minuten missbraucht. Mehr dazu in Kapitel 6.

2.7 Git – dein Sicherheitsnetz

Git speichert Schnappschüsse (englisch: *commits*) deines Projekts. Jeder Schnappschuss hat eine Nachricht, und du kannst jederzeit zu einem früheren Stand zurückkehren. Für Vibe Coding ist das Gold wert: Wenn die KI etwas kaputt macht, gehst du einfach zurück.

Einmalige Einrichtung

Beispiel 2.14: Git konfigurieren

```
PS> git config --global user.name "Dein Name"
PS> git config --global user.email "du@example.org"
PS> git config --global init.defaultBranch main
PS> git config --global --list
```

> **Gut zu wissen**
>
> **Zeilenenden:** Windows beendet Textzeilen mit zwei Zeichen (CRLF), Linux und macOS mit einem (LF). Git für Windows fragt bei der Installation, wie es damit umgehen soll; die Standardauswahl („Checkout Windows-style, commit Unix-style“) ist für den Anfang richtig. Wenn Git beim Committen „LF will be replaced by CRLF“ meldet, ist das eine Information, kein Fehler.

Das erste Repository

Beispiel 2.15: Erstes Repository mit erstem Commit

```
PS> mkdir "$HOME\vibe\projekte\git-test"; Set-Location "$HOME\vibe\projekte\git-test"
PS> git init
PS> "# Mein erstes Projekt" | Set-Content README.md -Encoding utf8
PS> git status
PS> git add README.md
PS> git commit -m "Erster Commit: README angelegt"
PS> git log --oneline
a1b2c3d Erster Commit: README angelegt
```

Der Kreislauf besteht aus drei Schritten:

1. Du änderst Dateien (**Arbeitsverzeichnis**).

2. Mit `git add` markierst du, welche Änderungen in den nächsten Schnappschuss sollen (**Staging-Bereich**).
3. Mit `git commit` speicherst du den Schnappschuss (**Repository**).

Die zehn Befehle, die du wirklich brauchst

Befehl	Wirkung
`git status`	Was hat sich geändert?
`git diff`	Zeige die Änderungen im Detail
`git add -A`	Alle Änderungen für den Commit vormerken
`git commit -m "Text"`	Schnappschuss speichern
`git log --oneline`	Verlauf kompakt anzeigen
`git restore datei`	Datei auf den letzten Commit zurücksetzen
`git switch -c name`	Neuen Zweig (Branch) anlegen und wechseln
`git switch main`	Zurück zum Hauptzweig
`git stash`	Änderungen kurz beiseitelegen
`git revert <id>`	Einen Commit durch einen Gegen-Commit rückgängig machen

Die .gitignore-Datei

Manche Dateien gehören nicht ins Repository: Keys, virtuelle Umgebungen, temporäre Dateien. Du schließt sie in einer Datei namens `.gitignore` aus. Der Inhalt ist unter Windows derselbe wie unter Linux:

Beispiel 2.16: Eine sinnvolle .gitignore für Python-Projekte

```
PS> @"
.venv/
__pycache__/
*.pyc
.env
*.env
*.log
"@ | Set-Content .gitignore -Encoding utf8
PS> git add .gitignore; git commit -m "gitignore hinzugefügt"
```

TIPP – Profi-Trick

Die Vibe-Coding-Regel Nr. 1: Committe **vor jeder größeren KI-Änderung**. Dann kostet jeder Fehlschlag nur einen Befehl: `git restore .` verwirft alle Änderungen seit dem letzten Commit. Wer beliebig experimentieren will, nutzt einen eigenen Branch: `git switch -c experiment`.

WARNUNG – Sicherheitsrisiko

`git restore .` und `git reset --hard` löschen nicht committete Änderungen **endgültig**. Prüfe vorher mit `git status` und `git diff`, was du verlierst.

2.8 Projektvorlage zum Wiederverwenden

Zum Abschluss ein kleines PowerShell-Skript, das neue Projekte mit der richtigen Grundstruktur anlegt. Du kannst es selbst abtippen oder dir von der KI erklären lassen.

Beispiel 2.17: Skript "neues-projekt.ps1" zum Anlegen neuer Projekte

```
# Legt ein neues Projekt mit Git, venv und .gitignore an.
param(
    [Parameter(Mandatory = $true)]
    [string]$Name
)
$ErrorActionPreference = "Stop"

# Nur einfache Namen erlauben: Buchstaben, Ziffern, _ und -, nicht leer.
# Das verhindert, dass z. B. "..\anderer-ordner" oder ein Laufwerkspfad
# wie "C:\temp" das vorgesehene Projekteverzeichnis verlassen.
if ($Name -notmatch '^[A-Za-z0-9][A-Za-z0-9_-]*$') {
    Write-Host "Fehler: '$Name' ist kein gültiger Projektname" -ForegroundColor Red
    Write-Host "(nur Buchstaben, Ziffern, _ und -)." -ForegroundColor Red
    exit 1
}
$projekteWurzel = Join-Path $HOME "vibe\projekte"
$ziel = Join-Path $projekteWurzel $Name
if (Test-Path $ziel) {
    Write-Host "Fehler: $ziel existiert bereits." -ForegroundColor Red
    exit 1
}

New-Item -ItemType Directory -Path $ziel | Out-Null
Set-Location $ziel
git init -q
py -m venv .venv
@(".venv/", "__pycache__/", "*.pyc", ".env", "*.env", "*.log") |
    Set-Content .gitignore -Encoding utf8
@("# $Name", "", "Kurzbeschreibung folgt.") | Set-Content README.md -Encoding utf8
git add -A
git commit -q -m "Projektgerüst angelegt"
Write-Host "Projekt angelegt: $ziel"
```

Speichere das Skript als `$HOME\bin\neues-projekt.ps1`:

Beispiel 2.18: Skriptdatei anlegen

```
PS> New-Item -ItemType Directory -Force "$HOME\bin" | Out-Null
PS> notepad "$HOME\bin\neues-projekt.ps1"        # Inhalt aus Beispiel 2.17 einfügen
```

> **Gut zu wissen**
>
> Anders als bei ausführbaren Programmen (`.exe`) reicht es unter Windows **nicht**, den Ordner mit dem Skript einfach in den Suchpfad (`PATH`) aufzunehmen: PowerShell sucht beim Eintippen eines bloßen Namens wie `neues-projekt` standardmäßig nur nach Endungen aus der Variable `PATHEXT`, und `.ps1` steht dort aus Sicherheitsgründen **nicht** mit drin. Der Aufruf `neues-projekt` würde die Datei also gar nicht finden, selbst wenn `bin` im `PATH` liegt. Die robuste Lösung ist eine kleine **Funktion in deinem PowerShell-Profil**, die das Skript gezielt mit vollem Pfad aufruft.

Beispiel 2.19: Aufruf über eine Funktion im PowerShell-Profil

```
PS> Test-Path $PROFILE                     # existiert deine Profildatei schon?
PS> New-Item -ItemType File -Force $PROFILE | Out-Null   # legt sie bei Bedarf an
PS> notepad $PROFILE
```

Füge in der geöffneten Profildatei diese Zeile ein und speichere:

Beispiel 2.20: Inhalt der Profildatei (eine Zeile ergänzen)

```
function neues-projekt { & "$HOME\bin\neues-projekt.ps1" @args }
```

Beispiel 2.21: Terminal neu starten und testen

```
# Terminal schließen und neu öffnen, damit das Profil neu geladen wird, dann:
PS> neues-projekt demo
Projekt angelegt: C:\Users\DEINNAME\vibe\projekte\demo
PS> neues-projekt ..\anderer-ordner
Fehler: '..\anderer-ordner' ist kein gültiger Projektname
(nur Buchstaben, Ziffern, _ und -).
```

`@args` reicht alle Argumente, die du der Funktion mitgibst, unverändert an das Skript weiter. Die Ausführungsrichtlinie (Kapitel 2.3) betrifft hier **nur deine Profildatei**, nicht jedes einzelne Skript, das du später so einbindest – ein weiterer Vorteil dieses Wegs gegenüber dem direkten PATH-Eintrag.

Erklärung der wichtigsten Zeilen: `param(...)` definiert die Argumente des Skripts, hier den Pflichtparameter `Name`. `$ErrorActionPreference = "Stop"` lässt das Skript bei Fehlern abbrechen, statt weiterzumachen. Die Zeile mit `-notmatch` prüft den Namen gegen ein **reguläres Muster** (nur Buchstaben, Ziffern, `_` und `-`, beginnend mit einem alphanumerischen Zeichen) und weist alles andere zurück – auch Namen mit `\`, `/` oder `..`, die sonst aus dem vorgesehenen Projektordner herausführen könnten, sowie vollständige Laufwerkspfade wie `C:\temp`. `Join-Path` baut Pfade zusammen, `Test-Path` prüft, ob etwas unter diesem Pfad existiert. `exit 1` beendet das Skript mit Fehlercode 1. Die Zeilen mit `[Environment]::...` ergänzen den **Benutzer-Suchpfad** um deinen `bin`-Ordner, ohne den Systempfad anzufassen.

> **TIPP – Profi-Trick**
>
> Diese Art Prüfung – **erst die Eingabe gegen ein enges, erlaubtes Muster validieren, dann erst damit arbeiten** – ist ein allgemeines Prinzip, das weit über dieses Skript hinaus gilt: Lieber genau festlegen, was erlaubt ist (Allowlist), als zu versuchen, einzelne gefährliche Zeichen zu verbieten (Denylist).

Übungen

> **Übung**
>
> 1. Lege mit Beispiel 2.4 deinen Arbeitsordner an und prüfe das Ergebnis mit `tree /F`.
> 2. Erstelle eine venv in `$HOME\vibe\experimente\test`, installiere `requests` mit `python -m pip install requests` (eindeutiger als ein bloßes `pip`, falls mehrere Python-Versionen installiert sind), erzeuge `requirements.txt` und lösche danach die venv – prüfe mit `Remove-Item -Recurse -Force .venv -WhatIf` erst, was gelöscht würde, bevor du `-WhatIf` entfernst. Stelle die venv aus `requirements.txt` wieder her.
> 3. Initialisiere **in einem eigens dafür angelegten Testordner** ein Git-Repository, lege drei Commits mit sinnvollen Nachrichten an und sieh dir den Verlauf mit `git log --oneline` an.
> 4. Ändere im selben Testordner eine Datei, schaue die Änderung mit `git diff` an und verwirf sie mit `git restore datei`. **Wichtig:** `git restore` macht nur uncommittete Änderungen rückgängig – übe das deshalb nur in diesem Testrepository, nie an Dateien mit echter, noch nicht gesicherter Arbeit.
> 5. Lege (ohne echten Key) die Datei `keys.ps1` an, schütze sie mit `icacls` und prüfe das Ergebnis mit `icacls $datei`.

6. Richte `neues-projekt` wie in Beispiel 2.18–2.21 über eine Profilfunktion ein und teste sowohl einen gültigen Namen als auch `..\test`.

Lösungshinweis

Zu 2: Nach dem Löschen von `.venv` legst du mit `py -m venv .venv` eine neue an, aktivierst sie und führst `python -m pip install -r requirements.txt` aus. Zu 5: In der Ausgabe von `icacls` darf nur dein Benutzerkonto mit `(R,W)` stehen. Zu 6: Der zweite Aufruf meldet „kein gültiger Projektname“ und bricht ab, bevor irgendein Ordner angelegt wird.

Zusammenfassung

Zusammenfassung

- Deine Umgebung besteht aus Windows Terminal mit PowerShell, Python, Editor, KI-Assistent und Git; `winget` installiert die Programme.
- Jedes Python-Projekt bekommt eine eigene virtuelle Umgebung (`py -m venv .venv`) – nie global installieren.
- Bei der Meldung zur Ausführungsrichtlinie hilft `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, nicht `Unrestricted`.
- VS Code ist der bequemste Einstieg; Neovim und Notepad sind legitime Alternativen.
- Beginne mit einem Chat-Assistenten; wechsle zu Editor-Integration oder Terminal-Agenten, wenn du Routine hast.
- API-Keys liegen in einer geschützten Datei oder einer Benutzervariable, nie im Code, nie im Repository und nie im Terminal-Verlauf.
- Git ist dein Sicherheitsnetz: committen vor jeder größeren KI-Änderung.

Kapitel 3: Der Vibe-Coding-Workflow

Lernziele

- Du kennst den Kreislauf aus Beschreiben, Erzeugen, Testen, Korrigieren und Sichern.
- Du zerlegst eine Idee in kleine, testbare Schritte.
- Du setzt die KI zum Debuggen ein und gibst ihr den richtigen Kontext.
- Du führst ein Lerntagebuch und eine Projekt-Notizdatei für die KI.

3.1 Der Grundkreislauf

Erfolgreiches Vibe Coding folgt immer demselben Rhythmus. Wer ihn verinnerlicht, vermeidet 90 Prozent aller Frustmomente:

1. **Ziel klären:** Was soll am Ende passieren? (Ein Satz.)
2. **Kleinen Schritt beschreiben:** Nicht „baue mir eine App“, sondern „lies eine Datei und zähle die Zeilen“.
3. **Code erzeugen lassen.**
4. **Ausführen und testen:** Tut es, was es soll?
5. **Bei Erfolg: sichern** (`git commit`). Bei Fehler: Meldung samt Kontext zurückgeben.
6. **Nächster kleiner Schritt.**

Der wichtigste Satz dieses Kapitels: **Je kleiner der Schritt, desto besser das Ergebnis.** Große Aufträge produzieren große, schwer prüfbare Codemengen, in denen sich Fehler verstecken. Kleine Aufträge liefern kleine Änderungen, die du verstehen und testen kannst.

Die Faustregel „Ein Schritt = ein Test“

Nach jedem Schritt solltest du mit einem einzigen Befehl prüfen können, ob er funktioniert hat. Wenn dir kein Test einfällt, ist der Schritt zu groß oder zu unklar.

Zu groß	Besser
„Baue eine Aufgabenverwaltung mit Login und Datenbank.“	„Schreibe ein Programm, das eine Aufgabe in eine Textdatei schreibt.“
„Mach das Skript schneller.“	„Miss, welcher Teil am längsten dauert, und zeige mir die drei langsamsten Stellen.“
„Repariere alle Fehler.“	„Hier ist die Fehlermeldung. Erkläre die Ursache, bevor du etwas änderst.“

3.2 Vor dem ersten Prompt: Plan und Projektnotiz

Bevor du Code erzeugen lässt, nimm dir fünf Minuten für einen Plan. Du kannst dir dabei von der KI helfen lassen, aber **entscheide selbst** über den Umfang.

Beispiel 3.1: Planungs-Prompt – erst planen, dann bauen

```
Ich möchte ein Kommandozeilen-Tool für Windows bauen, das doppelte Dateien in einem
Ordner findet. Ich bin Anfänger. Bitte schreibe noch KEINEN Code.
Schlage mir stattdessen vor:
1. eine minimale Version (MVP), die in einer Stunde machbar ist,
2. eine Liste von 5 kleinen Schritten, jeweils mit einem Testbefehl,
3. was ich bewusst weglasse.
```

Die Antwort sicherst du in einer Projektdatei. Diese Datei dient dir **und** der KI als Gedächtnis, denn Assistenten vergessen zwischen Gesprächen alles (und bei langen Gesprächen auch Details vom Anfang).

Beispiel 3.2: Projektnotiz PROJEKT.md als Gedächtnis

```
# Projekt: dupfinder

## Ziel
Findet doppelte Dateien in einem Ordner (gleicher Inhalt, egal welcher Name).

## Rahmen
- Sprache: Python 3, nur Standardbibliothek
- Läuft unter Windows in PowerShell: `py dupfinder.py ORDNER`
- Löscht NIE automatisch etwas

## Stand
- [x] Schritt 1: Dateien im Ordner auflisten
- [ ] Schritt 2: Hash pro Datei berechnen
- [ ] Schritt 3: Gleiche Hashes gruppieren
- [ ] Schritt 4: Ausgabe formatieren
## Entscheidungen
- Hash-Verfahren: SHA-256 (sicher genug, in der Standardbibliothek)
```

Diese Datei legst du ins Projektverzeichnis und committest sie mit. Viele Terminal-Agenten lesen sogar automatisch eine Projektdatei mit Regeln (zum Beispiel `CLAUDE.md` bei Claude Code oder `AGENTS.md` bei anderen Werkzeugen). Auch bei einem Chat-Assistenten kannst du sie einfach am Gesprächsbeginn einfügen.

> **TIPP – Profi-Trick**
>
> Beginne jedes neue Gespräch mit demselben Block: Projektziel, Sprache, Rahmenbedingungen („nur Standardbibliothek“, „keine neuen Abhängigkeiten ohne Rückfrage“), aktueller Stand. Das spart dir endlose Wiederholungen und verhindert, dass die KI plötzlich ein riesiges Framework einbaut.

3.3 Code erzeugen, lesen, ausführen

Schritt 1 am Beispiel dupfinder

Prompt:

Beispiel 3.3: Prompt für einen kleinen, klaren Schritt

```
Schreibe dupfinder.py. Das Programm bekommt einen Ordnerpfad als Argument und
gibt alle Dateien darin (rekursiv) mit Pfad und Größe in Bytes aus.
Nur Standardbibliothek. Wenn der Ordner nicht existiert, eine klare
Fehlermeldung ausgeben und mit Exit-Code 1 beenden.
```

Ergebnis (typisch):

Beispiel 3.4: dupfinder.py, Schritt 1 – Dateien auflisten

```
import sys
from pathlib import Path


def alle_dateien(ordner: Path):
    """Liefert alle regulären Dateien unterhalb von ordner."""
    for pfad in ordner.rglob("*"):
        if pfad.is_file():
            yield pfad


def main():
    if len(sys.argv) != 2:
        print("Benutzung: py dupfinder.py ORDNER", file=sys.stderr)
        sys.exit(2)
    ordner = Path(sys.argv[1])
    if not ordner.is_dir():
        print(f"Fehler: '{ordner}' ist kein Ordner.", file=sys.stderr)
        sys.exit(1)
    for datei in alle_dateien(ordner):
        print(f"{datei.stat().st_size:>12}  {datei}")


if __name__ == "__main__":
    main()
```

So liest du Code, ohne Programmierer zu sein

Du musst keine Zeile auswendig kennen. Lies von **oben nach unten** und achte auf diese Bausteine:

- `import` holt fertige Werkzeugkästen (hier `sys` für Argumente, `Path` für Dateipfade).
- `def name(...)` definiert eine **Funktion**, einen benannten Baustein. `main()` ist hier der Startpunkt.
- `if ...:` sind Bedingungen („wenn dies, dann das“). Der eingerückte Block darunter gehört dazu.
- `for ... in ...:` ist eine Schleife: etwas für jedes Element wiederholen.
- `print(...)` gibt Text aus; `file=sys.stderr` schreibt in den Fehlerkanal.
- `sys.exit(1)` beendet das Programm mit Exit-Code 1 (Fehler). Dieselbe Konvention kennst du von der Shell: 0 bedeutet Erfolg.

Wichtig bei Python: **Einrückung ist Syntax**. Vier Leerzeichen zeigen, was zu welchem Block gehört.

Ausführen und beobachten

Beispiel 3.5: Schritt 1 testen

```
PS> cd $HOME\vibe\projekte\dupfinder
PS> py dupfinder.py $HOME\vibe\notizen
         142  C:\Users\eugen\vibe\notizen\tagebuch.md
PS> py dupfinder.py C:\gibt\es\nicht
Fehler: 'C:\gibt\es\nicht' ist kein Ordner.
PS> $LASTEXITCODE
1
```

Du hast gerade drei Dinge getestet: den Normalfall, den Fehlerfall und den Exit-Code. Das ist **mehr Qualitätssicherung, als viele Einsteiger je betreiben**.

Danach folgt der Commit:

Beispiel 3.6: Funktionierenden Stand sichern

```
PS> git add -A
PS> git commit -m "dupfinder: Dateien rekursiv auflisten"
```

3.4 Iterieren: Änderungswünsche formulieren

Nach dem ersten Erfolg wird der Code Schritt für Schritt erweitert. Immer mit dem aktuellen Code im Gespräch und einer klaren Eingrenzung:

Beispiel 3.7: Erweiterung mit klarer Eingrenzung

```
Hier ist mein aktuelles dupfinder.py (siehe unten). Erweitere es um Schritt 2:
Berechne für jede Datei den SHA-256-Hash (in Blöcken von 1 MiB, damit große
Dateien nicht komplett in den Speicher geladen werden) und gib statt der
Größe den Hash aus. Ändere sonst nichts am Programm. Zeige mir nur die
geänderten Funktionen, nicht die ganze Datei.

[Code einfügen]
```

Die Formulierungen „Ändere sonst nichts“ und „Zeige nur die geänderten Funktionen“ sind kleine, aber mächtige Hebel: Sie verhindern, dass die KI unaufgefordert umbaut.

Änderungen prüfen mit git diff

Wenn du die Änderung übernommen hast, schau dir an, **was sich wirklich geändert hat**:

Beispiel 3.8: Änderungen der KI mit git diff kontrollieren

```
PS> git diff
@@ -5,6 +5,13 @@ from pathlib import Path
+import hashlib
+
+def sha256_von(datei: Path) -> str:
+    h = hashlib.sha256()
+    with datei.open("rb") as f:
+        for block in iter(lambda: f.read(1024 * 1024), b""):
+            h.update(block)
+    return h.hexdigest()
```

Zeilen mit `+` sind neu, Zeilen mit `-` wurden entfernt. Selbst als Anfänger erkennst du so schnell, ob die KI nur das Gewünschte getan hat oder auch Unerwartetes: gelöschte Funktionen, geänderte Dateinamen, neue Importe.

> **TIPP – Profi-Trick**
>
> Ein Blick auf `git diff --stat` zeigt dir in einer Zeile pro Datei, wie viel sich geändert hat. Wenn du nur „eine Funktion ergänzt“ hast erwarten lassen und 400 Zeilen geändert wurden, stimmt etwas nicht.

3.5 Debugging mit KI

Debugging heißt, Fehler zu finden und zu beheben. Es ist die Disziplin, in der KI-Assistenten besonders glänzen – wenn du ihnen die richtigen Informationen gibst.

Die vier Bestandteile einer guten Fehlermeldung an die KI

1. **Was wolltest du tun?** (Ziel)

2. **Was hast du getan?** (exakter Befehl)
3. **Was ist passiert?** (komplette Fehlermeldung, nicht nur die letzte Zeile)
4. **Welche Umgebung?** (Windows-Version, Python-Version, relevante Dateien)

Beispiel 3.9: Vorlage für eine Fehleranfrage

```
Ziel: dupfinder soll alle Dateien in $HOME\Bilder hashen.
Befehl: py dupfinder.py $HOME\Bilder
Fehler (komplett):
Traceback (most recent call last):
  File "dupfinder.py", line 24, in <module>
    main()
  File "dupfinder.py", line 20, in main
    h = sha256_von(datei)
PermissionError: [Errno 13] Permission denied: 'C:\\Users\\eugen\\Bilder\\geheim.jpg'
Umgebung: Windows 11, PowerShell 7, Python 3.13.
Erkläre mir zuerst die Ursache in zwei Sätzen. Schlage dann eine minimale
Korrektur vor, die das Programm bei nicht lesbaren Dateien nicht abstürzen lässt.
```

Du kannst die Fehlerausgabe direkt aus dem Terminal kopieren. Wenn du sie in eine Datei schreiben willst, hilft die Umleitung beider Kanäle:

Beispiel 3.10: Fehlerausgabe in eine Datei umleiten

```
PS> py dupfinder.py $HOME\Bilder > ausgabe.txt 2> fehler.txt
PS> py dupfinder.py $HOME\Bilder 2>&1 | Tee-Object lauf.log
```

Mit `2>` leitest du den Fehlerkanal um, mit `2>&1` führst du ihn mit der normalen Ausgabe zusammen. `Tee-Object` zeigt die Ausgabe am Bildschirm und schreibt sie zugleich in die Datei. In Windows PowerShell 5.1 speichert `>` Dateien als UTF-16; für Dateien, die du weiterverwenden willst, ist `| Set-Content -Encoding utf8` sicherer (siehe Kapitel 7).

Die Debug-Schleife

1. Fehlermeldung samt Kontext an die KI geben.
2. **Ursache erklären lassen**, bevor ein Fix kommt. Das verhindert blinde Pflaster.
3. Fix einspielen, erneut testen.
4. Tritt derselbe Fehler nach zwei Versuchen weiter auf: **stopp**. Wechsle die Strategie (siehe Abschnitt 3.7).

Eigene Diagnose-Ausgaben

Manchmal weißt du nicht, was im Programm vorgeht. Bitte die KI, **Diagnose-Ausgaben** einzubauen (`print` oder das Modul `logging`) und starte erneut. So siehst du, welche Werte wirklich vorliegen:

Beispiel 3.11: Diagnose-Ausgaben mit logging

```
import logging

logging.basicConfig(level=logging.DEBUG, format="%(levelname)s: %(message)s")

def verarbeite(datei):
    logging.debug("Verarbeite %s", datei)
    # ...
```

3.6 Kontext-Management

Ein KI-Assistent kann nur so viel „im Kopf“ behalten, wie in sein **Kontextfenster** passt, also die Menge an Text, die er pro Gespräch gleichzeitig berücksichtigt. Das Fenster ist heute oft sehr groß, aber nicht unendlich, und die Qualität sinkt, wenn Gespräche extrem lang und unübersichtlich werden.

Symptome eines überladenen Kontexts

- Die KI „vergisst“ Vorgaben vom Anfang (zum Beispiel „nur Standardbibliothek“).
- Sie wiederholt Fehler, die ihr schon erklärt wurden.
- Sie ändert Dinge, die du nicht angefasst haben wolltest.

Gegenmittel

Maßnahme	Wie
Neues Gespräch beginnen	Bei neuem Thema oder nach mehr als etwa 30 Nachrichten
Zusammenfassung anfordern	„Fasse Projektstand und Entscheidungen in 10 Zeilen zusammen“ – in die neue Sitzung einfügen
Projektnotiz pflegen	PROJEKT.md aktuell halten (Beispiel 3.2)
Nur Relevantes zeigen	Nicht das ganze Projekt, sondern die betroffene Datei und die Fehlermeldung
Struktur zeigen	`git ls-files` an die KI geben (nur Dateien, die Git verwaltet)

Beispiel 3.12: Projektstruktur kompakt für die KI ausgeben

```
PS> git ls-files
.gitignore
PROJEKT.md
README.md
dupfinder.py
tests/test_dupfinder.py
```

Dateien gezielt zeigen

Schicke nicht ganze Projekte hin, sondern gezielt Auszüge:

Beispiel 3.13: Gezielte Auszüge statt ganzer Dateien

```
PS> Get-Content dupfinder.py | Select-Object -Skip 14 -First 26   # Zeilen 15-40
PS> Select-String -Path dupfinder.py -Pattern "def "              # alle Funktionen
PS> git diff HEAD~1                                               # letzte Änderung
```

> **WARNUNG – Sicherheitsrisiko**
>
> Bevor du Dateien in einen Chat kopierst, prüfe, dass **keine Zugangsdaten** darin stehen: API-Keys, Passwörter, Tokens, private Adressen. Suche vorher mit `Select-String -Path datei -Pattern "key|token|passwort|secret"`.

3.7 Wenn nichts mehr geht: Notausgänge

Irgendwann steckst du fest: Die KI dreht sich im Kreis, jeder Fix erzeugt zwei neue Fehler. Hier sind deine Notausgänge, nach Aufwand sortiert:

1. **Zurück zum letzten guten Stand:** `git restore .` oder `git stash`. Danach den Schritt **kleiner** neu formulieren.
2. **Neues Gespräch:** Nur Projektnotiz, aktuelle Datei und die Fehlermeldung einfügen.
3. **Problem isolieren:** Ein winziges Testprogramm, das nur den Fehler reproduziert (die KI kann es schreiben).
4. **Anderen Assistenten fragen:** Eine zweite Meinung hilft oft.
5. **Verständnisfrage statt Reparaturbitte:** „Erkläre mir, was dieser Programmteil tun soll.“ Oft liegt das Problem in einem Missverständnis über die Aufgabe.
6. **Pause machen.** Ernsthaft.

3.8 Das Lerntagebuch

Eine Datei `$HOME\vibe\notizen\tagebuch.md` kostet zwei Minuten pro Sitzung und macht dich schnell besser:

Beispiel 3.14: Lerntagebuch-Eintrag

```
## 2026-10-23 – dupfinder
- Gebaut: Dateien auflisten, Hash berechnen
- Neu gelernt: `with open(...)` schließt Dateien automatisch
- Fehler: PermissionError -> mit try/except abgefangen
- Frage offen: Warum liest man in Blöcken statt alles auf einmal?
- Nächstes Mal: Gruppieren nach Hash
```

Die offene Frage ist der wertvollste Teil: Stelle sie in der nächsten Sitzung der KI. So wächst dein Verständnis, ohne dass du je ein Lehrbuch durcharbeiten musst.

Übungen

Übung

1. Plane ein eigenes kleines Tool (zum Beispiel „sortiere Downloads nach Dateityp“). Lass die KI nur einen Plan mit fünf Schritten und je einem Testbefehl erstellen – ohne Code.
2. Setze Schritt 1 deines Plans um. Teste Normal- und Fehlerfall und committe.
3. Baue absichtlich einen Fehler ein (zum Beispiel einen falschen Dateipfad) und formuliere die Fehleranfrage nach Beispiel 3.9.
4. Lass die KI eine Änderung machen und kontrolliere sie mit `git diff --stat` und `git diff`. Hat sie mehr geändert als gewünscht?
5. Lege `PROJEKT.md` für dein Tool an und beginne ein neues Gespräch mit dieser Datei.

Lösungshinweis

Zu 3: Eine gute Anfrage nennt Ziel, Befehl, die vollständige Fehlermeldung und die Umgebung und bittet zuerst um eine Erklärung der Ursache. Zu 4: Falls die KI zusätzliche Dateien oder Funktionen verändert hat, ergänze beim nächsten Mal „Ändere sonst nichts“ und prüfe den Diff erneut.

Zusammenfassung

Zusammenfassung

- Der Kreislauf: Ziel klären, kleinen Schritt beschreiben, erzeugen, testen, sichern, wiederholen.
- Ein Schritt = ein Test. Wenn du nicht testen kannst, ist der Schritt zu groß.
- Eine Projektnotiz (`PROJEKT.md`) ist das Gedächtnis von dir und der KI.
- Mit `git diff` prüfst du jede KI-Änderung; committe jeden funktionierenden Stand.
- Gute Fehleranfragen enthalten Ziel, Befehl, vollständige Fehlermeldung und Umgebung; lass zuerst die Ursache erklären.
- Halte den Kontext schlank: neue Gespräche, Zusammenfassungen, gezielte Auszüge.
- Notausgänge: zurücksetzen, neu starten, isolieren, zweite Meinung, Pause.

Kapitel 4: Prompting-Grundlagen für Code

Lernziele

- Du formulierst Anweisungen präzise, vollständig und überprüfbar.
- Du lieferst Kontext, ohne die KI zu überfluten.
- Du speist Fehlermeldungen und Logs richtig ein.
- Du nutzt die KI für Code-Reviews und Verständnisfragen.

4.1 Was einen guten Prompt ausmacht

Ein **Prompt** ist deine Anweisung an die KI. Die Qualität des Codes hängt stark von der Qualität des Prompts ab. Ein guter Code-Prompt beantwortet sechs Fragen:

Frage	Beispiel
Was soll gebaut werden?	„Ein Python-Skript, das Logdateien auswertet“
Wofür / warum?	„Ich will sehen, welche IP-Adressen am häufigsten fehlschlagen“
Womit (Technik)?	„Python 3, nur Standardbibliothek, läuft unter Windows 11“
Wie soll es sich verhalten?	„Aufruf: `py auswerten.py LOGDATEI`; Ausgabe als Tabelle“
Was nicht?	„Keine Dateien verändern, keine Netzwerkzugriffe“
Woran erkenne ich Erfolg?	„Bei dieser Beispieldatei soll Zeile X erscheinen“

Vorher / Nachher

Schwacher Prompt:

Beispiel 4.1: Ein zu vager Prompt

```
Mach mir ein Skript für meine Logs.
```

Starker Prompt:

Beispiel 4.2: Ein präziser Prompt

```
Schreibe ein Python-3-Skript (nur Standardbibliothek) für Windows.
Aufgabe: Es liest eine Logdatei (Pfad per Argument) und zählt, wie oft
pro IP-Adresse der Text "Failed login" vorkommt.
Ausgabe: die 10 häufigsten IP-Adressen als Tabelle (IP, Anzahl), absteigend.
Randfälle: Datei nicht lesbar -> verständliche Meldung und Exit-Code 1.
Nicht erlaubt: Dateien ändern, Netzwerkzugriffe, externe Pakete.
Erkläre am Ende in drei Sätzen, wie das Skript funktioniert.
```

Der zweite Prompt kostet 30 Sekunden mehr Tipparbeit und spart eine halbe Stunde Hin und Her.

4.2 Bewährte Muster

Muster 1: Rolle und Zielgruppe

„Erkläre mir das, als wäre ich Windows-Anwender ohne Programmierkenntnisse.“ Diese eine Zeile verändert die Erklärungen drastisch: weniger Fachbegriffe, mehr Vergleiche.

Muster 2: Erst verstehen, dann bauen

Beispiel 4.3: Rückfragen erzwingen

```
Bevor du Code schreibst: Stelle mir bis zu fünf Rückfragen, falls meine
Beschreibung etwas offen lässt. Wenn alles klar ist, sage "Alles klar".
```

Dieses Muster fängt Missverständnisse ab, bevor sie in Code stecken.

Muster 3: Beispiele geben

Zeige Eingabe und gewünschte Ausgabe, denn nichts ist eindeutiger:

Beispiel 4.4: Mit Beispiel-Ein- und Ausgabe arbeiten

```
Das Programm soll Dateinamen aufräumen. Beispiele:
  "Urlaub 2024 (final).JPG"  ->  "urlaub_2024_final.jpg"
  "  Mein Foto!!.png"         ->  "mein_foto.png"
Regeln: Kleinbuchstaben, Leerzeichen und Sonderzeichen zu "_", Endung klein.
```

Muster 4: Format vorgeben

„Zeige nur das Diff.“ – „Antworte mit einer Tabelle.“ – „Gib erst den Plan, dann den Code in einem einzigen Block.“

Muster 5: Grenzen setzen

Schreibe Einschränkungen ausdrücklich hin: „Keine neuen Abhängigkeiten.“ – „Ändere keine anderen Funktionen.“ – „Lösche nichts, ohne zu fragen.“

Muster 6: Selbstprüfung verlangen

Beispiel 4.5: Selbstprüfung anfordern

```
Prüfe deinen Code vor der Antwort auf: Randfälle (leere Datei, fehlende Rechte),
Pfade mit Leerzeichen und mögliche Datenverluste. Nenne mir, was du geprüft hast.
```

> **Gut zu wissen**
>
> Selbstprüfung verbessert Ergebnisse oft, ersetzt aber **nie** deinen eigenen Test. Die KI kann behaupten, geprüft zu haben, ohne den Code je auszuführen (sofern sie keine Ausführungsumgebung hat).

4.3 Kontext liefern: Was, wie viel, in welcher Form

Kontext ist das, was die KI über dein Projekt wissen muss. Zu wenig führt zu Raten, zu viel zu Verwirrung. Eine praktische Reihenfolge:

1. **Zielbeschreibung** (zwei Sätze)
2. **Umgebung** (Windows-Version, Sprache und Version, wichtige Pakete)
3. **Relevanter Code** (die betroffene Funktion oder Datei, nicht das ganze Projekt)

4. **Konkretes Problem oder Wunsch**
5. **Ausgabeformat**

Die Umgebung ermittelst du mit wenigen Befehlen:

Beispiel 4.6: Umgebungsinfos für die KI sammeln

```
PS> (Get-CimInstance Win32_OperatingSystem).Caption
PS> py --version
PS> pip list | Select-Object -First 20
PS> $env:PROCESSOR_ARCHITECTURE      # Prozessor-Architektur, z. B. AMD64 oder ARM64
```

Kopiere Code in **Codeblöcke**, damit Struktur und Einrückung erhalten bleiben. Bei Dateien mit Namen schreibe den Namen davor:

Beispiel 4.7: Mehrere Dateien sauber übergeben

```
Datei: dupfinder.py
(Inhalt)

Datei: tests/test_dupfinder.py
(Inhalt)
```

4.4 Fehlermeldungen richtig einspeisen

Die Fehlermeldung ist der wertvollste Hinweis, den die KI bekommen kann. Gib sie **vollständig und unverändert** weiter.

Eine Fehlermeldung lesen lernen

Python-Fehler (**Tracebacks**) liest du **von unten nach oben**: Die letzte Zeile nennt die Fehlerart und die Ursache, darüber steht der Weg dorthin.

Beispiel 4.8: Traceback richtig lesen

```
Traceback (most recent call last):
  File "auswerten.py", line 31, in <module>      <- 3. wo wurde gestartet
    main()
  File "auswerten.py", line 22, in main          <- 2. aus welcher Funktion
    zahlen = [int(z) for z in zeile.split()]
ValueError: invalid literal for int() with base 10: 'abc'   <- 1. DAS ist das Problem
```

Hier steht: Der Text „abc“ ließ sich nicht in eine ganze Zahl umwandeln. Die Ursache liegt in den **Daten** (eine Zeile enthält Text), nicht im Programm selbst.

PowerShell-Fehler

Beispiel 4.9: Typische Shell-Fehler und ihre Bedeutung

```
Die Ausführung von Skripts ist deaktiviert  -> Set-ExecutionPolicy (Kapitel 2.3)
Der Begriff "jq" wird nicht als Name erkannt -> Programm fehlt; Terminal neu öffnen
Zugriff verweigert / Access is denied         -> fehlende Rechte oder Datei in Benutzung
Unerwartetes Token "}" in Ausdruck            -> Klammerfehler im PowerShell-Skript
```

Logs und lange Ausgaben kürzen

Bei langen Logs sende nicht alles, sondern die Umgebung der ersten Fehlermeldung:

Beispiel 4.10: Relevante Logzeilen herausfiltern

```
PS> Get-WinEvent -LogName Application -MaxEvents 20 | Format-List TimeCreated, Message
PS> Select-String -Path lauf.log -Pattern "Traceback" -Context 3,10 | Select-Object -First 5
PS> Get-Content lauf.log -Tail 40
```

`-Context 3,10` zeigt drei Zeilen davor und zehn danach. Entferne vor dem Senden persönliche Daten wie Benutzernamen, IP-Adressen oder Tokens.

> **TIPP – Profi-Trick**
>
> Die schnellste Debug-Anfrage der Welt: Fehlermeldung kopieren, einfügen, dazuschreiben „Was bedeutet das, und was ist die wahrscheinlichste Ursache? Bitte erst erklären, noch nichts ändern."

4.5 Verständnisfragen – so lernst du nebenbei

Du lernst schneller, wenn du nicht nur reparieren, sondern **verstehen** lässt:

Frage	Zweck
„Erkläre diese Funktion Zeile für Zeile."	Verstehen
„Was passiert, wenn die Datei leer ist?"	Randfälle erkennen
„Warum hast du X statt Y gewählt?"	Alternativen kennenlernen
„Was ist der einfachste Weg, das zu testen?"	Testen lernen
„Welche Teile dieses Programms sind riskant?"	Risikobewusstsein
„Erkläre das mit einem Alltagsvergleich."	Intuition aufbauen

4.6 Code-Reviews durch KI

Ein **Code-Review** ist eine kritische Durchsicht von Code. Die KI ist hier ein guter, unermüdlicher Gegenleser, aber kein Ersatz für Tests.

Beispiel 4.11: Review-Prompt

```
Reviewe den folgenden Code als erfahrener Entwickler. Ich bin Anfänger,
erkläre also verständlich. Prüfe in dieser Reihenfolge:
1. Fehler und Randfälle (leere Eingaben, fehlende Rechte, Sonderzeichen)
2. Sicherheit (Eingaben, Dateipfade, Befehlsausführung, Geheimnisse)
3. Gefahr von Datenverlust
4. Lesbarkeit und Aufbau
Gib pro Fund: Zeile, Problem, Schweregrad (hoch/mittel/niedrig), Korrekturvorschlag.
Ändere den Code nicht, sondern berichte nur.
```

Der Hinweis „ändere den Code nicht" ist wichtig: Erst bewertest du die Funde, dann entscheidest du, was umgesetzt wird.

Review-Ergebnisse einordnen

- **Nicht blind übernehmen.** Frage bei jedem Fund: Ist das ein echtes Problem für mein Vorhaben?
- **Schweregrad hinterfragen:** KI übertreibt manchmal Kleinigkeiten und übersieht manchmal Größeres.
- **Zweite Meinung:** Lass einen anderen Assistenten über denselben Code schauen. Was beide nennen, ist meist wichtig.

- **Linter ergänzen:** Werkzeuge wie `PSScriptAnalyzer` (PowerShell) und `ruff` oder `pyflakes` (Python) finden Fehler **deterministisch**, also ohne Raterei.

Beispiel 4.12: Statische Prüfwerkzeuge laufen lassen

```
PS> Install-Module PSScriptAnalyzer -Scope CurrentUser
PS> Invoke-ScriptAnalyzer .\neues-projekt.ps1
PS> py -m pip install ruff            # in einer venv
PS> ruff check dupfinder.py
PS> py -m py_compile dupfinder.py     # reine Syntaxprüfung
```

4.7 Tests als Prompt-Hilfe

Ein **Test** ist ein kleines Programm, das prüft, ob dein Code das Richtige tut. Lass dir Tests schreiben, **bevor** du Funktionen erweiterst. Dann bemerkst du sofort, wenn eine KI-Änderung etwas kaputt macht.

Beispiel 4.13: Ein einfacher Test mit unittest (Standardbibliothek)

```
import unittest
from namen import aufraeumen


class TestAufraeumen(unittest.TestCase):
    def test_leerzeichen_und_gross(self):
        self.assertEqual(aufraeumen("Urlaub 2024 (final).JPG"),
                         "urlaub_2024_final.jpg")
    def test_fuehrende_leerzeichen(self):
        self.assertEqual(aufraeumen("  Mein Foto!!.png"), "mein_foto.png")


if __name__ == "__main__":
    unittest.main()
```

Beispiel 4.14: Tests ausführen

```
PS> py -m unittest -v
test_fuehrende_leerzeichen (test_namen.TestAufraeumen) ... ok
test_leerzeichen_und_gross (test_namen.TestAufraeumen) ... ok
Ran 2 tests in 0.001s
OK
```

Das Prompt-Muster dazu:

Beispiel 4.15: Tests vor der Änderung anfordern

```
Schreibe zuerst unittest-Tests für die Funktion aufraeumen(), die die Beispiele
oben und mindestens vier Randfälle abdecken (leerer String, nur Sonderzeichen,
Umlaute, sehr langer Name). Danach ändere die Funktion so, dass alle Tests bestehen.
```

Übungen

> **Übung**
>
> 1. Verbessere den Prompt „Mach ein Skript, das meine Fotos sortiert“ nach den sechs Fragen aus 4.1.
> 2. Schreibe einen Prompt mit Beispiel-Ein- und Ausgabe (Muster 3) für ein Skript, das Dateien nach Erweiterung in Unterordner verschiebt.

3. Erzeuge mit `py -c "print(int('abc'))"` einen Fehler und lass ihn dir von der KI erklären. Lies den Traceback vorher selbst von unten nach oben.
4. Bitte die KI, ein eigenes Skript zu reviewen (Beispiel 4.11). Bewerte jeden Fund: berechtigt, übertrieben oder falsch?
5. Lass dir Tests für eine kleine Funktion schreiben und führe sie mit `py -m unittest -v` aus.

Lösungshinweis

Zu 1: Ein guter Prompt nennt Ordnerpfad, Sortierkriterium (Datum aus EXIF oder Dateidatum?), Zielstruktur, was bei Namenskonflikten passiert, dass nie gelöscht wird, und ein Beispiel. Zu 3: Der Traceback endet mit `ValueError: invalid literal for int() with base 10: 'abc'`.

Zusammenfassung

Zusammenfassung

- Gute Prompts klären Was, Wofür, Womit, Wie, Was nicht und Woran du Erfolg erkennst.
- Muster: Rückfragen erzwingen, Beispiele geben, Format vorgeben, Grenzen setzen, Selbstprüfung verlangen.
- Kontext in sinnvoller Reihenfolge liefern: Ziel, Umgebung, relevanter Code, Problem, Format.
- Fehlermeldungen vollständig weitergeben; Tracebacks von unten nach oben lesen.
- Lass dir erklären statt nur reparieren – so lernst du nebenbei.
- Code-Reviews durch KI sind nützlich, aber ergänze sie durch Linter und eigene Tests.
- Tests vor Änderungen schützen dich vor stillen Rückschritten.

Kapitel 5: Drei Praxisprojekte

Lernziele

- Du baust ein vollständiges CLI-Tool (Duplikate finden) mit Argumenten, Fehlerbehandlung und Tests.
- Du baust eine kleine Webanwendung (HTML, CSS, JavaScript) und erweiterst sie optional mit Flask.
- Du baust ein Automatisierungsskript mit Trockenlauf und zeitgesteuerter Ausführung.
- Du wendest in jedem Projekt den Workflow aus Kapitel 3 an.

Jedes Projekt folgt demselben Schema: **Ziel, Plan, Schritte mit Prompts, Test, Commit, Erweiterungen**. Die Prompts sind Vorschläge – formuliere sie gern um. Der Code sind typische Ergebnisse, die dir als Vergleich dienen; deine KI wird etwas anderes liefern, und das ist völlig in Ordnung, solange deine Tests bestehen.

Projekt 1: dupfinder – ein CLI-Tool in Python

Ziel

Ein Programm, das in einem Ordner doppelte Dateien (gleicher Inhalt) findet und anzeigt. Es **löscht nie** etwas, sondern berichtet nur.

Schritt 0: Projekt anlegen

Beispiel 5.1: Projekt anlegen

```
PS> neues-projekt dupfinder          # Skript aus Beispiel 2.17
PS> Set-Location $HOME\vibe\projekte\dupfinder
PS> mkdir tests
PS> code PROJEKT.md                 # Ziel und Rahmen eintragen (Vorlage: Beispiel 3.2)
```

Schritt 1 bis 2: Auflisten und Hashen

Die Schritte kennst du aus Kapitel 3 (Beispiele 3.3 und 3.7). Falls du sie dort nicht gebaut hast, hole das jetzt nach. Danach hast du eine Datei `dupfinder.py`, die Dateien rekursiv durchgeht und SHA-256-Hashes berechnet.

Gut zu wissen

Warum ein Hash? Ein Hash ist ein „digitaler Fingerabdruck“ eines Dateiinhalts. Zwei Dateien mit demselben SHA-256 haben mit praktisch absoluter Sicherheit denselben Inhalt, egal wie sie heißen. So vergleichst du Dateien, ohne sie Byte für Byte nebeneinanderzulegen.

Schritt 3: Gruppieren und effizient werden

Prompt:

Beispiel 5.2: Prompt für das Gruppieren

```
Erweitere dupfinder.py. Gruppiere Dateien mit gleichem Hash. Optimierung:
Berechne den Hash nur für Dateien, deren Größe mit mindestens einer anderen
Datei übereinstimmt. Gib nur Gruppen mit mindestens 2 Dateien aus.
Nicht lesbare Dateien: Warnung auf stderr, aber weitermachen.
Füge Argparse hinzu mit --min-groesse (Bytes, Standard 1).
Zeige die vollständige Datei.
```

Ein typisches Ergebnis (vollständiges Programm):

Beispiel 5.3: dupfinder.py – vollständige Fassung

```
"""dupfinder - findet doppelte Dateien (gleicher Inhalt) in einem Ordner."""
import argparse
import hashlib
import os
import sys
from collections import defaultdict
from pathlib import Path
BLOCK = 1024 * 1024  # 1 MiB pro Lesevorgang


def alle_dateien(ordner: Path):
    """Liefert alle regulären Dateien (keine Verknüpfungen) unterhalb von ordner.

    Nutzt os.walk() mit followlinks=False statt Path.rglob(), weil rglob()
    standardmäßig auch in per Verknüpfung erreichbare Unterordner hineinschaut:
    Eine normale Datei, die nur über einen solchen Ordner erreichbar ist,
    würde sonst trotz der Einschränkung "keine Verknüpfungen" mit aufgenommen.
    Unter Windows betrifft das vor allem symbolische Verknüpfungen (mklink)
    und Verzeichnisverbindungen (mklink /J); normale Verknüpfungen (.lnk)
    sind für Python ohnehin reguläre Dateien und keine Symlinks.
    """
    for wurzel, unterordner, dateien in os.walk(ordner, followlinks=False):
        for name in dateien:
            pfad = Path(wurzel) / name
            if pfad.is_file() and not pfad.is_symlink():
                yield pfad


def sha256_von(datei: Path) -> str:
    h = hashlib.sha256()
    with datei.open("rb") as f:
        for block in iter(lambda: f.read(BLOCK), b""):
            h.update(block)
    return h.hexdigest()


def finde_duplikate(ordner: Path, min_groesse: int = 1):
    """Gibt eine Liste von Gruppen zurück; jede Gruppe ist eine Liste von Pfaden."""
    nach_groesse = defaultdict(list)
    for datei in alle_dateien(ordner):
        try:
            groesse = datei.stat().st_size
        except OSError as fehler:
            print(f"Warnung: {datei}: {fehler}", file=sys.stderr)
            continue
        if groesse >= min_groesse:
            nach_groesse[groesse].append(datei)
    nach_hash = defaultdict(list)
    for groesse, dateien in nach_groesse.items():
        if len(dateien) < 2:
            continue  # einzigartige Größe -> kann kein Duplikat sein
        for datei in dateien:
            try:
                nach_hash[(groesse, sha256_von(datei))].append(datei)
            except OSError as fehler:
                print(f"Warnung: {datei}: {fehler}", file=sys.stderr)

    return [sorted(gruppe) for gruppe in nach_hash.values() if len(gruppe) > 1]


def menschlich(bytes_: int) -> str:
    wert = float(bytes_)
```

```
    for einheit in ("B", "KiB", "MiB", "GiB"):
        if wert < 1024:
            return f"{wert:.1f} {einheit}"
        wert /= 1024
    return f"{wert:.1f} TiB"


def main() -> int:
    parser = argparse.ArgumentParser(description="Findet doppelte Dateien.")
    parser.add_argument("ordner", type=Path, help="zu durchsuchender Ordner")
    parser.add_argument("--min-groesse", type=int, default=1,
                        help="Mindestgröße in Bytes (Standard: 1)")
    args = parser.parse_args()

    if not args.ordner.is_dir():
        print(f"Fehler: '{args.ordner}' ist kein Ordner.", file=sys.stderr)
        return 1

    gruppen = finde_duplikate(args.ordner, args.min_groesse)
    verschwendet = 0
    for nr, gruppe in enumerate(gruppen, start=1):
        groesse = gruppe[0].stat().st_size
        verschwendet += groesse * (len(gruppe) - 1)
        print(f"Gruppe {nr}: {len(gruppe)} Dateien, je {menschlich(groesse)}")
        for pfad in gruppe:
            print(f"  {pfad}")
    print(f"\n{len(gruppen)} Gruppen, vermeidbarer Platz: {menschlich(verschwendet)}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Schritt 4: Testen mit echten Testdaten

Teste **nie** zuerst an deinen echten Daten. Erzeuge eine kleine Testumgebung:

Beispiel 5.4: Testordner mit bekannten Duplikaten erzeugen

```
PS> $t = "$env:TEMP\dup-test"
PS> New-Item -ItemType Directory -Force "$t\unter" | Out-Null
PS> "gleicher Inhalt" | Set-Content "$t\a.txt" -Encoding utf8
PS> "gleicher Inhalt" | Set-Content "$t\unter\b.txt" -Encoding utf8
PS> "anderer Inhalt"  | Set-Content "$t\c.txt" -Encoding utf8
PS> py dupfinder.py $t
Gruppe 1: 2 Dateien, je 17.0 B
  C:\Users\DEINNAME\AppData\Local\Temp\dup-test\a.txt
  C:\Users\DEINNAME\AppData\Local\Temp\dup-test\unter\b.txt

1 Gruppen, vermeidbarer Platz: 17.0 B
```

Das Ergebnis stimmt: a.txt und b.txt sind Duplikate, c.txt nicht. (Die genaue Byte-Zahl hängt von PowerShell-Version und Zeilenende ab.) Jetzt noch ein automatischer Test, damit das auch nach späteren Änderungen so bleibt:

Beispiel 5.5: tests/test_dupfinder.py

```
import sys
import tempfile
import unittest
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from dupfinder import finde_duplikate  # noqa: E402
class TestDuplikate(unittest.TestCase):
    def test_findet_duplikate(self):
        with tempfile.TemporaryDirectory() as tmp:
            p = Path(tmp)
            (p / "a.txt").write_text("gleich")
            (p / "b.txt").write_text("gleich")
            (p / "c.txt").write_text("anders")
            gruppen = finde_duplikate(p)
            self.assertEqual(len(gruppen), 1)
            self.assertEqual({x.name for x in gruppen[0]}, {"a.txt", "b.txt"})

    def test_leerer_ordner(self):
        with tempfile.TemporaryDirectory() as tmp:
            self.assertEqual(finde_duplikate(Path(tmp)), [])
    def test_min_groesse(self):
        with tempfile.TemporaryDirectory() as tmp:
            p = Path(tmp)
            (p / "a.txt").write_text("xx")
            (p / "b.txt").write_text("xx")
            self.assertEqual(finde_duplikate(p, min_groesse=100), [])
if __name__ == "__main__":
    unittest.main()
```

> **Gut zu wissen**
>
> Symbolische Verknüpfungen unter Windows können nur mit aktiviertem **Entwicklermodus** oder mit Administratorrechten angelegt werden (`mklink` bzw. `Path.symlink_to()`). Schlägt der folgende Zusatztest mit einem Berechtigungsfehler fehl, fehlt eine der beiden Voraussetzungen – überspringe ihn dann einfach.

Beispiel 5.5b: Zusätzlicher Test mit Verknüpfungen (Entwicklermodus oder Administratorrechte erforderlich)

```
def test_verknuepfung_auf_datei_wird_uebersprungen(self):
    with tempfile.TemporaryDirectory() as tmp:
        p = Path(tmp)
        (p / "original.txt").write_text("gleich")
        (p / "verweis.txt").symlink_to(p / "original.txt")
        (p / "kopie.txt").write_text("gleich")
        gruppen = finde_duplikate(p)
        self.assertEqual(len(gruppen), 1)
        self.assertEqual({x.name for x in gruppen[0]}, {"original.txt", "kopie.txt"})
```

Beispiel 5.6: Tests ausführen und committen

```
PS> py -m unittest discover -s tests -v
...
Ran 3 tests in 0.004s
OK
PS> git add -A
PS> git commit -m "dupfinder: Gruppieren, argparse, Tests"
```

> **Gut zu wissen**
>
> In **Windows PowerShell 5.1** verursachen `&&` und `||` als Befehlsverkettung einen Parserfehler – sie funktionieren erst ab **PowerShell 7**. Schreibe Befehle deshalb wie oben in getrennte Zeilen, oder nutze, wenn du gezielt „nur bei Erfolg“ ausführen willst, `if ($?) { ... }` (siehe Beispiel 5.6b).

Beispiel 5.6b: Bedingte Ausführung ohne &&, kompatibel mit PowerShell 5.1 und 7

```
PS> git add -A
PS> if ($?) { git commit -m "dupfinder: Gruppieren, argparse, Tests" }
```

Schritt 5: Komfortabel aufrufen

Beispiel 5.7: dupfinder wie ein echtes Kommandozeilenprogramm installieren

```
PS> Set-Content "$HOME\bin\dupfinder.ps1" 'py "$HOME\vibe\projekte\dupfinder\dupfinder.py" @args'
```

Wie schon bei `neues-projekt` (Kapitel 2.18) reicht es nicht, `bin` im `PATH` zu haben: `.ps1` fehlt standardmäßig in `PATHEXT`, der bloße Aufruf `dupfinder` würde die Datei nicht finden. Ergänze deshalb dieselbe Art Funktion in deinem Profil (`notepad $PROFILE`):

Beispiel 5.7b: Aufruf über eine Funktion im PowerShell-Profil

```
function dupfinder { & "$HOME\bin\dupfinder.ps1" @args }
```

Beispiel 5.7c: Terminal neu starten und testen

```
# Terminal schließen und neu öffnen, dann:
PS> dupfinder $HOME\Downloads --min-groesse 1048576
```

Erweiterungen zum Üben

- Ausgabe als JSON mit `--json` (nützlich für andere Programme, `jq` kann sie auswerten).
- Option `--ignoriere ".git"`, um bestimmte Ordner zu überspringen.
- **Sicher** löschen: Duplikate in einen Papierkorb-Ordner **verschieben**, nicht löschen, und zwar nur mit ausdrücklicher Bestätigung.

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Programm, das Dateien löscht, ist der klassische Ort für Datenverlust. Wenn du die Löschfunktion baust: Standard ist immer der **Trockenlauf** (nur anzeigen), echtes Handeln nur mit einer Option wie `--ausfuehren`, und teste das zuerst in `$env:TEMP`.

Projekt 2: Ein kleines Web-Projekt

Ziel

Eine Aufgabenliste im Browser: Aufgaben hinzufügen, abhaken, löschen, Daten bleiben nach dem Neuladen erhalten. **Eine einzige Datei**, ohne Server und ohne Installation.

Plan

1. Seite mit Eingabefeld und Liste (HTML und CSS)

2. Aufgaben hinzufügen (JavaScript)
3. Abhaken und Löschen
4. Speichern im Browser (`localStorage`)

Wenn du das HTML-Grundgerüst schon aus früheren Unterlagen kennst: Hier zählt nur, wie die Teile zusammenspielen. **HTML** liefert den Inhalt, **CSS** das Aussehen, **JavaScript** das Verhalten.

Prompt

Beispiel 5.8: Prompt für die Aufgabenliste

```
Baue eine Aufgabenliste als einzelne Datei index.html (HTML, CSS und JavaScript
in einer Datei, keine externen Bibliotheken, kein Netzwerkzugriff).
Funktionen: Aufgabe eingeben und mit Enter oder Button hinzufügen; Aufgabe
per Klick abhaken (durchgestrichen); Löschen-Button pro Aufgabe; Speichern
im localStorage. Leere Eingaben ignorieren. Oberfläche auf Deutsch, schlicht
und gut lesbar, auch auf dem Smartphone. Erkläre am Ende kurz den Aufbau.
```

Typisches Ergebnis:

Beispiel 5.9: index.html – Aufgabenliste (eine Datei)

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Aufgabenliste</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 32rem; margin: 2rem auto;
           padding: 0 1rem; color: #1f2933; }
    h1 { color: #0b6e4f; }
    form { display: flex; gap: .5rem; margin-bottom: 1rem; }
    input { flex: 1; padding: .6rem; font-size: 1rem; }
    button { padding: .6rem 1rem; font-size: 1rem; cursor: pointer; }
    ul { list-style: none; padding: 0; }
    li { display: flex; align-items: center; gap: .5rem; padding: .5rem 0;
         border-bottom: 1px solid #dde3e8; }
    li span { flex: 1; cursor: pointer; }
    li.fertig span { text-decoration: line-through; color: #8a949e; }
  </style>
</head>
<body>
  <h1>Meine Aufgaben</h1>
  <form id="formular">
    <input id="eingabe" placeholder="Neue Aufgabe ..." autocomplete="off">
    <button type="submit">Hinzufügen</button>
  </form>
  <ul id="liste"></ul>

  <script>
    const SCHLUESSEL = "aufgaben-v1";
    let aufgaben = [];
    try {
      aufgaben = JSON.parse(localStorage.getItem(SCHLUESSEL)) || [];
    } catch (e) {
      aufgaben = [];
    }

    const liste = document.getElementById("liste");
    const formular = document.getElementById("formular");
    const eingabe = document.getElementById("eingabe");
    function speichern() {
      localStorage.setItem(SCHLUESSEL, JSON.stringify(aufgaben));
    }

    function zeichnen() {
      liste.innerHTML = "";
      aufgaben.forEach((a, index) => {
        const li = document.createElement("li");
        if (a.fertig) li.className = "fertig";
        const text = document.createElement("span");
        text.textContent = a.text;           // textContent: sicher gegen Code-Einschleusung
        text.onclick = () => { a.fertig = !a.fertig; speichern(); zeichnen(); };
        const loeschen = document.createElement("button");
        loeschen.textContent = "Löschen";
        loeschen.onclick = () => { aufgaben.splice(index, 1); speichern(); zeichnen(); };
        li.append(text, loeschen);
        liste.append(li);
      });
    }

    formular.addEventListener("submit", (ereignis) => {
      ereignis.preventDefault();
      const text = eingabe.value.trim();
```

```
      if (!text) return;
      aufgaben.push({ text: text, fertig: false });
      eingabe.value = "";
      speichern();
      zeichnen();
    });

    zeichnen();
  </script>
</body>
</html>
```

Ausprobieren

Du kannst die Datei direkt im Browser öffnen oder einen lokalen Mini-Server starten:

Beispiel 5.10: Seite lokal ansehen

```
PS> Set-Location $HOME\vibe\projekte\aufgabenliste
PS> Start-Process .\index.html                      # öffnet die Datei im Standardbrowser
PS> py -m http.server 8000 --bind 127.0.0.1         # alternativ: Server, nur lokal erreichbar
# Browser: http://localhost:8000
```

`--bind 127.0.0.1` ist hier bewusst von Anfang an dabei: Ohne diese Option lauscht der Server standardmäßig auf **allen** Netzwerkschnittstellen und ist damit auch für andere Geräte im selben Netz erreichbar – in einem vertrauten Heimnetz meist unkritisch, in einem fremden Netz (Café, Hotel, Konferenz) aber ein unnötiges Risiko für diese bewusst einfach gehaltene Übungsanwendung.

Beispiel 5.11: Zum Vergleich – der ungefilterte Aufruf (bewusst, mit Erklärung)

```
PS> py -m http.server 8000
# Jetzt auch von anderen Geräten im selben Netz erreichbar. Fragt die
# Windows-Firewall nach, erlaube höchstens "Private Netzwerke", nie
# "Öffentliche Netzwerke".
```

> **TIPP – Profi-Trick**
>
> Öffne im Browser die **Entwicklerwerkzeuge** mit `F12`. Im Reiter „Konsole“ siehst du JavaScript-Fehler, im Reiter „Anwendung“ (oder „Speicher“) siehst du den `localStorage`. Kopiere Fehlermeldungen aus der Konsole in deine Anfrage an die KI.

Teste wie ein Nutzer

Probiere bewusst Unfug aus: leere Eingabe, ein sehr langer Text, Sonderzeichen wie `<b>fett</b>` (es muss als Text erscheinen, nicht als Formatierung), Seite neu laden. Das Letzte ist ein klassischer Sicherheitstest: Würde das Programm den Text als HTML interpretieren, könnte man Schadcode einschleusen (**XSS**, siehe Kapitel 6). Deshalb nutzt der Code `textContent` statt `innerHTML` für Nutzertexte.

Optional: Mit Flask zu einem echten Server

Wenn Daten nicht nur im Browser, sondern auf einem Server liegen sollen, brauchst du ein **Backend**. **Flask** ist ein kleines Python-Framework dafür.

Beispiel 5.12: Flask installieren

```
PS> Set-Location $HOME\vibe\projekte\aufgabenliste
PS> py -m venv .venv; .\.venv\Scripts\Activate.ps1
(.venv) PS> pip install flask
```

Beispiel 5.13: app.py – minimaler Flask-Server mit JSON-Schnittstelle

```
from flask import Flask, jsonify, request, send_from_directory

app = Flask(__name__)
aufgaben = []          # nur im Speicher: nach Neustart weg (für den Anfang okay)


@app.get("/")
def startseite():
    return send_from_directory(".", "index.html")


@app.get("/api/aufgaben")
def liste():
    return jsonify(aufgaben)
@app.post("/api/aufgaben")
def neu():
    daten = request.get_json(silent=True) or {}
    text = str(daten.get("text", "")).strip()
    if not text or len(text) > 200:
        return jsonify(fehler="Text fehlt oder ist zu lang"), 400
    aufgaben.append({"text": text, "fertig": False})
    return jsonify(aufgaben[-1]), 201
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```

Beispiel 5.14: Server starten und mit curl testen

```
(.venv) PS> py app.py                 # Terminal 1: Server läuft (Strg+C beendet ihn)
# Zweites Terminal öffnen:
PS> Invoke-RestMethod -Method Post -Uri http://localhost:5000/api/aufgaben `
      -ContentType "application/json" -Body '{"text":"Milch kaufen"}'
text         fertig
----         ------
Milch kaufen  False
PS> Invoke-RestMethod http://localhost:5000/api/aufgaben
```

> **Gut zu wissen**
>
> In Windows PowerShell 5.1 ist `curl` ein Alias für `Invoke-WebRequest` und verhält sich anders als das Linux-Programm. Das echte Programm erreichst du mit `curl.exe`; für Web-Abfragen in PowerShell ist `Invoke-RestMethod` ohnehin bequemer.

> **WARNUNG – Sicherheitsrisiko**
>
> Starte Flask niemals mit `debug=True` auf einem Rechner, der aus dem Netz erreichbar ist: Der Debugger erlaubt das Ausführen von Code. Für den echten Betrieb im Internet brauchst du einen Produktionsserver (zum Beispiel waitress unter Windows oder gunicorn unter Linux, jeweils hinter einem Reverse-Proxy) und weitere Absicherung – das ist ein eigenes Thema.

Projekt 3: Ein Automatisierungsskript

Ziel

Ein Skript, das deinen Ordner `$HOME\Downloads` aufräumt: Dateien wandern nach Typ in Unterordner (`Bilder`, `Dokumente`, `Archive`, `Sonstiges`). Es hat einen **Trockenlauf** als Standard und läuft auf Wunsch automatisch jeden Tag.

Plan

1. Dateien nach Endung einer Kategorie zuordnen
2. Trockenlauf: nur anzeigen, was passieren würde
3. Echtes Verschieben, ohne vorhandene Dateien zu überschreiben
4. Protokoll
5. Zeitsteuerung

Prompt

Beispiel 5.15: Prompt für das Aufräumskript

```
Schreibe aufraeumen.py (Python 3, nur Standardbibliothek) für Windows.
Es räumt einen Ordner auf (Argument, Standard $HOME\Downloads), indem es Dateien
direkt in diesem Ordner nach Endung in Unterordner verschiebt:
Bilder (jpg jpeg png gif webp svg), Dokumente (pdf txt odt docx md csv),
Archive (zip tar gz xz 7z), Sonstiges (alles andere).
Wichtig: Standard ist ein Trockenlauf, der nur ausgibt, was passieren würde.
Erst mit --ausfuehren wird verschoben. Nie etwas überschreiben: Bei Namens-
konflikt "name (2).endung" verwenden. Unterordner und versteckte Dateien
(Name beginnt mit Punkt) nicht anfassen. Jede Aktion in ein Logfile schreiben.
```

Typisches Ergebnis:

Beispiel 5.16: aufraeumen.py

```
"""aufraeumen - sortiert Dateien eines Ordners nach Typ in Unterordner."""
import argparse
import logging
import os
import shutil
import sys
from pathlib import Path
KATEGORIEN = {
    "Bilder": {"jpg", "jpeg", "png", "gif", "webp", "svg"},
    "Dokumente": {"pdf", "txt", "odt", "docx", "md", "csv"},
    "Archive": {"zip", "tar", "gz", "xz", "7z"},
}


def kategorie_von(datei: Path) -> str:
    endung = datei.suffix.lower().lstrip(".")
    for name, endungen in KATEGORIEN.items():
        if endung in endungen:
            return name
    return "Sonstiges"


def freier_name(ziel: Path) -> Path:
    """Reserviert atomar einen freien Namen; hängt bei Bedarf (2), (3) ... an.

    Reserviert heißt: Die Funktion legt am zurückgegebenen Pfad bereits eine
    leere Platzhalterdatei an, bevor sie zurückkehrt. Das schließt die Lücke
    zwischen "ist der Name frei?" und "jetzt verschieben": Zwischen beiden
    Schritten könnte sonst eine andere Datei genau diesen Namen belegen.
    os.O_EXCL sorgt dafür, dass ein bereits vorhandener Eintrag zuverlässig
    als belegt erkannt wird, auch wenn Path.exists() ihn unter Umständen
    übersieht (etwa bei einer defekten Verknüpfung).
    """
    kandidat = ziel
    zaehler = 2
    while True:
        try:
            fd = os.open(kandidat, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
            os.close(fd)
            return kandidat
        except FileExistsError:
            kandidat = ziel.with_name(f"{ziel.stem} ({zaehler}){ziel.suffix}")
            zaehler += 1


def main() -> int:
    parser = argparse.ArgumentParser(description="Ordner nach Dateityp aufräumen.")
    parser.add_argument("ordner", nargs="?", type=Path,
                        default=Path.home() / "Downloads")
    parser.add_argument("--ausfuehren", action="store_true",
                        help="wirklich verschieben (Standard: nur anzeigen)")
    args = parser.parse_args()

    if not args.ordner.is_dir():
        print(f"Fehler: '{args.ordner}' ist kein Ordner.", file=sys.stderr)
        return 1

    log_pfad = Path.home() / "vibe" / "notizen" / "aufraeumen.log"
    log_pfad.parent.mkdir(parents=True, exist_ok=True)
    logging.basicConfig(filename=log_pfad, level=logging.INFO,
                        format="%(asctime)s %(message)s")

    anzahl = 0
```

```
    for datei in sorted(args.ordner.iterdir()):
        if not datei.is_file() or datei.name.startswith("."):
            continue
        ziel_ordner = args.ordner / kategorie_von(datei)
        if args.ausfuehren:
            ziel_ordner.mkdir(exist_ok=True)
            try:
                # freier_name() reserviert den Namen atomar (siehe Kommentar dort).
                ziel = freier_name(ziel_ordner / datei.name)
                shutil.move(str(datei), str(ziel))  # überschreibt die eigene Platzhalterdatei
            except OSError as fehler:   # z. B. Datei ist gerade geöffnet
                print(f"Übersprungen: {datei.name}: {fehler}", file=sys.stderr)
                continue
            logging.info("verschoben: %s -> %s", datei, ziel)
        else:
            # Im Trockenlauf nur den voraussichtlichen Namen anzeigen, ohne
            # eine Platzhalterdatei anzulegen oder den Zielordner zu erstellen.
            vorschau = ziel_ordner / datei.name
            zaehler = 2
            while vorschau.exists():
                vorschau = vorschau.with_name(f"{datei.stem} ({zaehler}){datei.suffix}")
                zaehler += 1
            ziel = vorschau
        print(f"{'VERSCHOBEN' if args.ausfuehren else 'würde verschieben'}: "
              f"{datei.name} -> {ziel_ordner.name}/{ziel.name}")
        anzahl += 1

    if not args.ausfuehren:
        print(f"\nTrockenlauf: {anzahl} Dateien. Mit --ausfuehren wirklich verschieben.")
    return 0
if __name__ == "__main__":
    sys.exit(main())
```

Gut zu wissen

Warum die Platzhalterdatei? Zwischen „der Name ist frei" und „jetzt verschieben" kann eine andere Datei genau diesen Namen belegen (ein sogenanntes Zeitfenster- oder Race-Condition-Problem). `os.open(..., os.O_CREAT | os.O_EXCL)` legt den Namen **in einem einzigen, nicht unterbrechbaren Schritt** als leere Datei an und schlägt fehl, wenn er bereits existiert. Erst danach verschiebt `shutil.move()` die echte Datei dorthin und überschreibt dabei gezielt die eigene, gerade erst angelegte Platzhalterdatei. Im Trockenlauf entfällt diese Reservierung bewusst: Es wird nichts angelegt oder verändert, die Namensvorschau kann sich deshalb theoretisch bis zum echten Lauf noch ändern.

Sicher testen

Beispiel 5.17: Test im Wegwerf-Ordner statt in den echten Downloads

```
PS> $t = "$env:TEMP\dl-test"
PS> New-Item -ItemType Directory -Force $t | Out-Null; Set-Location $t
PS> "foto.JPG","bericht.pdf","archiv.tar.gz","notiz.xyz",".versteckt" |
      ForEach-Object { New-Item $_ -ItemType File | Out-Null }
PS> py $HOME\vibe\projekte\aufraeumen\aufraeumen.py $t
würde verschieben: archiv.tar.gz -> Archive/archiv.tar.gz
würde verschieben: bericht.pdf -> Dokumente/bericht.pdf
...
PS> py $HOME\vibe\projekte\aufraeumen\aufraeumen.py $t --ausfuehren
PS> tree $t /F
```

Prüfe: Wurde `.versteckt` in Ruhe gelassen? (Das Skript erkennt „versteckt“ am Punkt im Namen, nicht am Windows-Attribut „Ausgeblendet“.) Landete `foto.JPG` bei den Bildern? Lege danach erneut eine `foto.JPG` an und starte noch einmal mit `--ausfuehren`: Heißt die neue Datei jetzt `foto (2).JPG`, und

hat die alte überlebt? (Windows unterscheidet Groß- und Kleinschreibung bei Dateinamen nicht, deshalb funktioniert dieser Test mit nur einer Datei.) Erst wenn alles stimmt, probiere es an einer **Kopie** deiner echten Downloads.

Zeitsteuerung mit der Windows-Aufgabenplanung

Windows bringt dafür die **Aufgabenplanung** (englisch: Task Scheduler) mit. Du erreichst sie über das Startmenü („Aufgabenplanung“) oder bequem per PowerShell. Eine Aufgabe, die nur unter deinem Konto läuft, braucht keine Administratorrechte.

Wichtig: Registriere die Aufgabe zuerst **ohne** `--ausfuehren`. So kannst du sie gefahrlos mit `Start-ScheduledTask` testen, bevor sie jemals eine echte Datei verschiebt:

Beispiel 5.18: Tägliche Aufgabe registrieren (zunächst als Trockenlauf)

```
PS> $python = (Get-Command py).Source
PS> $skript = "$HOME\vibe\projekte\aufraeumen\aufraeumen.py"
PS> $aktion = New-ScheduledTaskAction -Execute $python -Argument "`"$skript`""
PS> $zeit   = New-ScheduledTaskTrigger -Daily -At 18:00
PS> $regeln = New-ScheduledTaskSettingsSet -StartWhenAvailable
PS> Register-ScheduledTask -TaskName "Downloads aufräumen" -Action $aktion `
      -Trigger $zeit -Settings $regeln
```

`-StartWhenAvailable` holt einen verpassten Lauf nach, wenn der Rechner um 18 Uhr aus war. Ohne `--ausfuehren` schreibt das Skript nur eine Vorschau ins Logfile (`$HOME\vibe\notizen\aufraeumen.log`) – es verschiebt nichts.

Beispiel 5.19: Trockenlauf-Aufgabe gefahrlos testen

```
PS> Get-ScheduledTask -TaskName "Downloads aufräumen" | Get-ScheduledTaskInfo
PS> Start-ScheduledTask -TaskName "Downloads aufräumen"   # Trockenlauf, verschiebt nichts
PS> Get-Content "$HOME\vibe\notizen\aufraeumen.log" -Tail 20
```

Weil die Aktion oben **kein** `--ausfuehren` enthält, verschiebt `Start-ScheduledTask` hier wirklich nur eine Vorschau – das ist bewusst so gewählt, damit du die Aufgabe gefahrlos von Hand anstoßen kannst, ohne auf 18 Uhr zu warten. Erst wenn das Log über mehrere Tage sinnvoll aussieht, schaltest du bewusst auf echten Betrieb um:

Beispiel 5.19b: Erst nach kontrollierter Sichtung auf echten Betrieb umstellen

```
PS> $aktionEcht = New-ScheduledTaskAction -Execute $python -Argument "`"$skript`" --ausfuehren"
PS> Set-ScheduledTask -TaskName "Downloads aufräumen" -Action $aktionEcht
```

Die Aufgabe entfernst du jederzeit mit:

Beispiel 5.19c: Aufgabe entfernen

```
PS> Unregister-ScheduledTask -TaskName "Downloads aufräumen" -Confirm:$false
```

> **Gut zu wissen**
>
> Die klassische Kommandozeilen-Alternative ist `schtasks`, die grafische die Aufgabenplanung selbst. Beide legen dieselben Aufgaben an. Unter Linux übernehmen cron oder systemd-Timer diese Rolle.

> **WARNUNG – Sicherheitsrisiko**

Ein **automatisch** laufendes Skript, das Dateien verschiebt, ist riskanter als ein von Hand gestartetes. Lass die Trockenlauf-Aufgabe oben einige Tage mitlaufen (von Hand mit `Start-ScheduledTask` angestoßen oder über den täglichen Trigger) und lies das Logfile, **bevor** du wie in Beispiel 5.19b auf `--ausfuehren` umstellst.

Übungen

Übung

1. Erweitere `dupfinder` um `--json`: Die Ausgabe soll eine JSON-Liste von Gruppen sein. Prüfe sie mit `| jq .`
2. Ändere die Aufgabenliste so, dass jede Aufgabe ein Datum bekommt. Lass die KI nur die nötigen Änderungen zeigen und prüfe sie mit `git diff`.
3. Gib `aufraeumen.py` eine Option `--rueckgaengig`, die anhand des Logfiles die letzten Verschiebungen umkehrt. Entwirf zuerst gemeinsam mit der KI den Plan (Kapitel 3.2) und teste im Wegwerf-Ordner.
4. Baue die Flask-Anwendung so aus, dass Aufgaben in einer Datei `aufgaben.json` gespeichert werden und einen Neustart überleben.
5. Lass die KI alle drei Projekte nach Beispiel 4.11 reviewen und bewerte die Funde.

Lösungshinweis

Zu 3: Das Logfile enthält die Zeilen „verschoben: Quelle -> Ziel“; die Umkehr verschiebt Ziel zurück zu Quelle. Wichtig sind Prüfungen, ob die Quelle inzwischen belegt ist, und der Trockenlauf als Standard. Zu 4: Beim Speichern zuerst in eine temporäre Datei schreiben und dann umbenennen, damit ein Absturz die Datei nicht halb geschrieben zurücklässt.

Zusammenfassung

Zusammenfassung

- Alle drei Projekte folgten demselben Weg: Plan, kleine Schritte, Test im Wegwerf-Umfeld, Commit.
- Tools, die Dateien verändern, bekommen immer einen Trockenlauf als Standard.
- Web-Projekte testest du wie ein misstrauischer Nutzer: leere, lange und seltsame Eingaben.
- Nutzertexte gehören mit `textContent` statt `innerHTML` in die Seite.
- Zeitgesteuerte Skripte erst als Trockenlauf beobachten, dann scharf schalten.
- Eigene Tests (`unittest`) halten das Verhalten stabil, wenn die KI später etwas ändert.

Kapitel 6: Sicherheit und Qualität

Lernziele

- Du gehst mit API-Keys und Zugangsdaten sicher um und erkennst Leaks.
- Du prüfst KI-Code systematisch auf typische Sicherheitsprobleme.
- Du kennst die Grundlagen zu Lizenzen und Datenschutz.
- Du weißt, wann du KI-Code nicht vertrauen darfst.

6.1 Warum Sicherheit hier besonders wichtig ist

Beim Vibe Coding entsteht viel Code schnell, und du prüfst ihn weniger genau, als wenn du ihn selbst geschrieben hättest. Das ist das Kernrisiko. Studien und Erfahrungsberichte zeigen immer wieder, dass KI-generierter Code Schwachstellen enthalten kann, weil Modelle Muster aus öffentlichem Code übernehmen, darunter auch unsichere. Das heißt nicht, dass du es lassen sollst, sondern dass du **eine Prüfroutine** brauchst.

Die drei größten Risikoquellen für Einsteiger:

1. **Geheimnisse an falscher Stelle** (Keys im Code, im Chat oder im Repository)
2. **Ungeprüfte Befehlsausführung** (KI-Befehle mit Administratorrechten, `Remove-Item -Recurse -Force`, `iex`)
3. **Unsicherer Code, der „funktioniert“** (fehlende Eingabeprüfung, offene Server)

6.2 API-Keys und Geheimnisse

Die goldenen Regeln

Regel	Warum
Keys nie im Quellcode	Code wird kopiert, geteilt, committet
Keys nie in Chats einfügen	Gesprächsinhalte können gespeichert werden
Keys nie committen	Git vergisst nichts, auch nach dem Löschen
Je Projekt eigener Key	Bei Verlust drehst du nur einen ab
Ausgabenlimit setzen	Begrenzt den Schaden bei Missbrauch
Rechte minimal	Nur die nötigen Berechtigungen vergeben

Keys in Programmen verwenden

Das Programm liest den Key aus der **Umgebungsvariable**, die du wie in Beispiel 2.12 bereitgestellt hast:

Beispiel 6.1: Key aus der Umgebung lesen, nie im Code speichern

```
import os
import sys

key = os.environ.get("ANTHROPIC_API_KEY")
if not key:
    print("Fehler: ANTHROPIC_API_KEY ist nicht gesetzt.", file=sys.stderr)
    sys.exit(1)
```

Das Programm gibt den Key niemals aus und schreibt ihn nicht ins Log. Prüfe das nach, wenn die KI Diagnose-Ausgaben eingebaut hat.

Leck-Suche vor jedem Commit

Beispiel 6.2: Nach Geheimnissen im Projekt suchen (nur im Git-Repository ausführen)

```
PS> git grep --untracked -n -i -E "(api[_-]?key|secret|token|passwort|password)[[:space:]]*[=:]"
PS> git diff --cached | Select-String -Pattern "key|token|secret"
PS> git status --short | Select-String "\.env"
```

Diese drei Zeilen decken unterschiedliche, aber begrenzte Fälle ab: Die erste durchsucht den **Inhalt** verfolgter und nicht verfolgter, aber nicht ignorierter Dateien nach dem Muster. Die zweite prüft, was bereits für den nächsten Commit vorgemerkt ist. Die dritte zeigt nur, **ob eine Datei namens** `.env` im Arbeitsverzeichnis auftaucht – sie durchsucht **nicht deren Inhalt** und findet auch kein anderweitig benanntes Geheimnis. Wenn Treffer erscheinen, prüfe jeden einzelnen.

> **Gut zu wissen**
>
> Diese Suche ist eine **Heuristik**: Sie findet nur Geheimnisse, die genau diesem Textmuster entsprechen, und durchsucht standardmäßig keine von `.gitignore` ausgeschlossenen Dateien vollständig. Ein „kein Treffer" ist also kein Beweis, dass keine Geheimnisse im Projekt stecken. Für wichtige Projekte ergänze ein spezialisiertes Werkzeug, das Repositories systematischer durchsucht (zum Beispiel `gitleaks` oder `trufflehog`). Frage deine KI nach aktuellen Empfehlungen und prüfe die Projektseite.

Was tun, wenn ein Key geleakt ist?

> **WARNUNG – Sicherheitsrisiko**
>
> **Sofortmaßnahmen bei einem geleakten Key:**
>
> 1. **Key im Anbieter-Portal sofort widerrufen** und einen neuen erzeugen. Das ist Schritt 1, vor allem anderen.
> 2. Nutzung und Kosten im Konto prüfen, Auffälligkeiten melden.
> 3. Key aus Code und Historie entfernen. Das Löschen in einem neuen Commit **reicht nicht**, denn die Historie enthält den Key weiter. Das Umschreiben der Historie ist nachrangig, weil der alte Key ohnehin ungültig sein muss.
> 4. Ursache beheben (`.gitignore`, Umgebungsvariable) und künftig vor jedem Commit prüfen.

6.3 Gefährliche Befehle erkennen

Terminal-Agenten und Chat-Antworten schlagen Befehle vor. **Du bist der Sicherheitsmechanismus.** Lerne diese Warnzeichen:

Muster	Gefahr
`Remove-Item -Recurse -Force`, `rd /s /q`, `del /s` mit Variablen oder Platzhaltern	löscht ganze Verzeichnisse, ohne Papierkorb
„Als Administrator ausführen“ bei unbekannten Befehlen	volle Systemrechte
Heruntergeladener Code, der sofort per `iex` (Invoke-Expression) läuft	führt Fremdcode ungeprüft aus
`Set-ExecutionPolicy Unrestricted` oder `icacls ... /grant Everyone:F`	schaltet Schutz ab bzw. öffnet Rechte für alle
`format`, `diskpart`, `Clear-Disk`	kann Datenträger löschen
`git push --force`, `git reset --hard`	zerstört Historie oder lokale Arbeit
`pip install` unbekannter Pakete	kann Schadpakete enthalten (siehe 6.6)
Änderungen an der Registrierung (`reg add`, `HKLM:`), am `$PROFILE`, an Defender-Ausnahmen	verändert Sicherheit oder Systemverhalten

Die Drei-Fragen-Prüfung vor jedem Befehl

1. **Verstehe ich, was der Befehl tut?** Wenn nein: Lass ihn dir Teil für Teil erklären.
2. **Was ist das Schlimmste, das passieren kann?** Wenn die Antwort „Daten weg“ lautet: Backup oder Test in `/tmp`.
3. **Kann ich es zuerst trocken ausprobieren?** Viele Programme haben dafür Optionen.

Beispiel 6.3: Trockenläufe und sichere Gewohnheiten

```
PS> robocopy Quelle Ziel /E /L          # /L = nur auflisten, nichts kopieren
PS> Remove-Item datei.txt -WhatIf       # Trockenlauf: zeigt, was passieren würde
PS> Remove-Item datei.txt -Confirm      # fragt vor dem Löschen nach
PS> Get-ChildItem ordner\*.tmp          # erst ansehen, was der Platzhalter trifft ...
PS> Remove-Item ordner\*.tmp            # ... dann löschen
```

> **TIPP – Profi-Trick**
>
> Bitte die KI bei jedem riskanten Befehl: „Erkläre jeden Teil dieses Befehls und nenne das schlimmste mögliche Ergebnis.“ Und: Starte Agenten nur in Projektordnern, nicht im Benutzerverzeichnis (siehe 2.5).

Isolation: Wegwerf-Umgebungen

Wenn du etwas Riskantes ausprobieren möchtest, tue es in einer **Wegwerf-Umgebung**: einer virtuellen Maschine, einem Container oder einem eigenen Benutzerkonto ohne wichtige Daten. Unter Windows ist **Windows Sandbox** der einfachste Weg: Es startet ein sauberes Windows in einem Fenster, und beim Schließen ist alles gelöscht.

> **Gut zu wissen**
>
> Windows Sandbox braucht **Administratorrechte zum Aktivieren** (danach läuft sie als normaler Nutzer) und ist nur in den Editionen **Pro, Enterprise und Education** verfügbar, nicht in Windows Home. Ist eine dieser Voraussetzungen nicht erfüllt, sind Hyper-V (ebenfalls Pro/Enterprise/Education) oder ein zweites lokales Benutzerkonto ohne wichtige Daten (*Einstellungen → Konten → Weitere Benutzer*) die Alternativen. **WSL ist dafür kein gleichwertiger Ersatz**, sobald Datei- oder Netzwerk-Integration zwischen Windows und der Linux-Umgebung aktiv ist (Standardeinstellung): Dateien im WSL-Dateisystem sind dann auch von Windows aus erreichbar, und ein „Wegwerfen" der Umgebung löscht nicht automatisch alles, was währenddessen auf der Windows-Seite verändert wurde.

Beispiel 6.4: Windows Sandbox aktivieren (einmalig, Administrator-Terminal, nur Pro/Enterprise/Education)

```
PS> Enable-WindowsOptionalFeature -Online -FeatureName "Containers-DisposableClientVM" -All
# Danach neu starten und "Windows-Sandbox" im Startmenü öffnen.
```

6.4 Typische Schwachstellen in KI-Code

Du musst kein Sicherheitsexperte werden, aber diese Muster solltest du kennen. Lass die KI gezielt danach suchen (Beispiel 4.11, Punkt 2).

Befehlseinschleusung (Command Injection)

Wenn Nutzereingaben in einen Shell-Befehl einfließen, kann ein Angreifer eigene Befehle anhängen.

Beispiel 6.5: Unsicher und sicher – Befehle aus Eingaben bauen

```
import subprocess

host = input("Host: ")      # Nutzer tippt:  localhost & echo GEHACKT

# UNSICHER: Die Eingabe wird von der Shell (cmd) interpretiert!
subprocess.run(f"ping -n 1 {host}", shell=True)

# SICHER: Argumente als Liste, keine Shell
subprocess.run(["ping", "-n", "1", host], check=False)
```

Merke: `shell=True` **mit Nutzereingaben ist fast immer falsch.** Ebenso gefährlich sind `eval()` und `exec()` mit fremden Daten.

Pfad-Tricks (Path Traversal)

Ein Eingabewert wie `..\..\Windows\System32` kann aus dem vorgesehenen Ordner ausbrechen:

Beispiel 6.6: Pfade absichern

```
from pathlib import Path

BASIS = Path("C:/Daten").resolve()

def sicherer_pfad(name: str) -> Path:
    pfad = (BASIS / name).resolve()
    if BASIS not in pfad.parents and pfad != BASIS:
        raise ValueError("Pfad außerhalb des erlaubten Ordners")
    return pfad
```

SQL-Einschleusung

Wenn du mit Datenbanken arbeitest: Baue SQL nie durch Textverkettung, sondern mit **Platzhaltern**.

Beispiel 6.7: Parametrisierte SQL-Abfrage mit sqlite3

```
import sqlite3

verbindung = sqlite3.connect("daten.db")
name = input("Name: ")

# UNSICHER: f"SELECT * FROM nutzer WHERE name = '{name}'"
zeilen = verbindung.execute("SELECT * FROM nutzer WHERE name = ?", (name,)).fetchall()
```

Cross-Site-Scripting (XSS)

Nutzertexte, die ungefiltert als HTML in eine Seite gelangen, können Schadcode ausführen. In JavaScript: `textContent` statt `innerHTML`. In Flask-Templates schützt die automatische Maskierung, solange du `|safe` nicht leichtfertig verwendest.

Weitere Dauerbrenner

- **Offene Server:** Ein Dienst auf `0.0.0.0` ist aus dem ganzen Netz erreichbar. Standard: `127.0.0.1`.
- **Schwache oder fehlende Anmeldung:** Eigene Login-Systeme sind schwer. Nutze bewährte Bibliotheken oder gar keine eigene Anmeldung.
- **Passwörter selbst speichern:** Niemals im Klartext. Wenn überhaupt, nutze bewährte Verfahren (zum Beispiel `argon2`, `bcrypt`), und frage dich zuerst, ob du das wirklich selbst bauen musst.
- **Fehlende Fehlerbehandlung:** Abstürze, die Zustände halb geändert hinterlassen.
- **Zu weite Rechte:** Dateien, auf die „Jeder“ Vollzugriff hat (`Everyone:F`), und Dienste mit Administratorrechten.

6.5 Qualitätssicherung in der Praxis

Eine Mindest-Routine, bevor du ein Programm „fertig“ nennst:

1. **Läuft es?** Normalfall, Fehlerfall und Randfälle ausprobiert.
2. **Tests** vorhanden und grün (`py -m unittest discover`).
3. **Linter** ohne ernste Meldungen (`ruff`, `PSScriptAnalyzer`).
4. `git diff` **gelesen:** Nichts Unerwartetes dabei.
5. **Geheimnis-Suche** durchgeführt (Beispiel 6.2).
6. **Abhängigkeiten geprüft:** Jedes Paket in `requirements.txt` ist bekannt und nötig.
7. **README** erklärt Zweck, Installation und Aufruf.
8. **Backup oder Git-Stand** vorhanden, bevor das Programm echte Daten anfasst.

Beispiel 6.8: Prüfroutine als kleines Skript

```
# pruefe.ps1 - Mindestprüfung vor einem Commit
# Voraussetzung: wird im Wurzelverzeichnis eines Git-Repositorys ausgeführt.
$fehler = 0
Write-Host "== Tests =="
py -m unittest discover -s tests
if ($LASTEXITCODE -ne 0) { $fehler = 1 }

Write-Host "== Linter =="
if (Get-Command ruff -ErrorAction SilentlyContinue) {
    ruff check .
    if ($LASTEXITCODE -ne 0) { $fehler = 1 }
} else {
    Write-Host "Hinweis: ruff nicht installiert, Linter-Schritt übersprungen."
}

Write-Host "== Geheimnisse (Heuristik, kein vollständiger Nachweis) =="
git grep --untracked -n -i -E `
    "(api[_-]?key|secret|token|password)[[:space:]]*[=:][[:space:]]*['""][^'""]{8,}" -- ":!*.md"
switch ($LASTEXITCODE) {
    0 { Write-Host "Möglicher Fund! Bitte prüfen."; $fehler = 1 }
    1 { Write-Host "Kein Treffer bei diesem Muster." }
    default {
        Write-Host "Fehler beim Durchsuchen (Exit-Code $LASTEXITCODE)"
        Write-Host "— z. B. kein Git-Repository hier?"
        $fehler = 1
    }
}

if ($fehler -eq 0) {
    Write-Host "Alles in Ordnung (im Rahmen dieser Prüfungen)."
} else {
    Write-Host "Es gibt Probleme."
}
exit $fehler
```

> **Gut zu wissen**
>
> `git grep` meldet über seinen Exit-Code drei Fälle: `0` = Treffer gefunden, `1` = kein Treffer, ein Wert größer als `1` = ein Fehler ist aufgetreten (zum Beispiel, weil der Ordner gar kein Git-Repository ist). Die `switch`-Anweisung oben unterscheidet diese drei Fälle bewusst, damit ein Fehler beim Durchsuchen nicht fälschlich als „keine Geheimnisse gefunden" durchgeht. Beachte außerdem: Diese Regex-Suche ist eine **Heuristik**, kein Beweis der Geheimnisfreiheit – sie findet nur Muster, die dem vorgegebenen Format entsprechen, durchsucht standardmäßig keine von Git ignorierten Dateien wie `.env` vollständig und kann andere Geheimnisformen übersehen. Für wichtige Projekte ergänze einen dedizierten Secret-Scanner (siehe Kapitel 6.2).

6.6 Abhängigkeiten und Lieferkette

Jedes Paket, das du installierst, ist fremder Code mit deinen Rechten. Zwei Probleme begegnen dir bei KI-Code besonders:

- **Erfundene Paketnamen:** Die KI schlägt ein Paket vor, das es nicht gibt. Angreifer registrieren solche Namen mitunter, um Schadcode zu verteilen („Slopsquatting“). Prüfe vor jeder Installation, ob das Paket real, bekannt und gepflegt ist.
- **Tippfehler-Pakete:** Namen, die bekannten Paketen ähneln (`reqeusts` statt `requests`).

Beispiel 6.9: Ein Paket vor der Installation prüfen

```
PS> pip index versions requests          # existiert es, welche Versionen?
PS> pip download requests --no-deps -d $env:TEMP\pk   # nur laden, nichts installieren
PS> py -m pip show requests         # nach der Installation: Autor, Seite, Lizenz
```

Schaue außerdem auf der Projektseite nach: Wie alt, wie verbreitet, wer pflegt es? Bevorzuge die **Standardbibliothek**, wo sie ausreicht. Weniger Abhängigkeiten bedeuten weniger Risiko. Du kannst die KI ausdrücklich dazu auffordern: „Nur Standardbibliothek“.

WARNUNG – Sicherheitsrisiko

Installiere niemals Pakete oder Skripte blind, weil „die KI es gesagt hat“. Besonders Einzeiler, die ein heruntergeladenes Skript sofort per `iex` (Invoke-Expression) ausführen, sind ein Sicherheitsrisiko. Lade das Skript stattdessen herunter, lies es (oder lass es dir erklären) und führe es erst dann aus. Bei heruntergeladenen Installern prüfst du den Fingerabdruck mit `Get-FileHash` und vergleichst ihn mit der Angabe des Herstellers. Ignoriere SmartScreen-Warnungen nicht.

6.7 Lizenzfragen

Wem gehört KI-generierter Code, und darfst du ihn verwenden? Die Rechtslage ist je nach Land im Fluss, und dieses Handbuch ersetzt keine Rechtsberatung. Diese Punkte solltest du kennen:

- **Urheberrecht an KI-Ausgaben:** In vielen Rechtsordnungen (auch in Deutschland) setzt ein urheberrechtlich geschütztes Werk eine persönliche geistige Schöpfung eines Menschen voraus. Rein maschinell erzeugter Code ist deshalb unter Umständen nicht geschützt. Dein eigener kreativer Beitrag (Planung, Auswahl, Anpassung) kann eine Rolle spielen.
- **Nutzungsbedingungen des Anbieters:** Jeder Dienst regelt, was du mit den Ausgaben tun darfst. Lies sie bei kommerziellem Einsatz.
- **Übernommener Code:** Modelle können Codeschnipsel erzeugen, die bekanntem Open-Source-Code ähneln. Bei größeren Abschnitten kann eine Lizenz greifen. Manche Dienste bieten Filter oder Quellenhinweise.
- **Open-Source-Lizenzen deiner Abhängigkeiten:** MIT und BSD sind sehr freizügig, die GPL verlangt unter Umständen Offenlegung abgeleiteter Werke. Prüfe die Lizenz jedes eingesetzten Pakets (`pip show`).
- **Eigene Lizenz wählen:** Wenn du Code veröffentlichst, lege eine `LICENSE`-Datei bei. Ohne Lizenz darf ihn rechtlich niemand weiterverwenden.

Gut zu wissen

Praxis-Empfehlung: Für private Werkzeuge und Lernprojekte sind Lizenzfragen kaum relevant. Sobald du Code **verkaufst, an Kunden gibst oder in einer Firma einsetzt**, kläre die Lage mit dem Arbeitgeber oder einer Rechtsberatung.

6.8 Datenschutz

Alles, was du in einen KI-Dienst eingibst, wird an einen externen Anbieter übertragen. Deshalb gilt:

Gib nie ein, was nicht nach außen gehören darf:

- Passwörter, Keys, Tokens, private Schlüssel (`.ssh\id_*`)

- Personenbezogene Daten Dritter (Kundenlisten, Namen mit Adressen, Gesundheitsdaten)
- Betriebsgeheimnisse und vertraulichen Firmencode, sofern dein Arbeitgeber das nicht ausdrücklich erlaubt
- Interne Netzwerkdaten (IP-Pläne, Hostnamen, Logdateien mit Nutzerdaten)

Sicher arbeiten mit echten Daten

- **Anonymisiere** Beispieldaten: Ersetze Namen, Adressen und Nummern durch erfundene Werte.
- **Schicke Strukturen statt Inhalte:** „Eine CSV mit den Spalten Name, Datum, Betrag“ genügt oft, um Code zu schreiben.
- **Erzeuge Testdaten**, am besten mit dem Programm selbst.
- **Prüfe Einstellungen:** Viele Anbieter erlauben, die Verwendung von Gesprächen zum Training einzuschränken. Schaue in die Datenschutzeinstellungen.
- **Lokale Modelle:** Für sehr sensible Daten gibt es Modelle, die auf deinem eigenen Rechner laufen (zum Beispiel über Ollama oder llama.cpp). Sie sind meist weniger leistungsfähig, aber deine Daten verlassen den Rechner nicht.

Beispiel 6.10: Testdaten für eine Beispiel-CSV selbst erzeugen (keine echten Daten nötig)

```
PS> $zeilen = @(
      [pscustomobject]@{ name = "Max Muster"; datum = "2026-01-15"; betrag = "42,50" }
      [pscustomobject]@{ name = "Erika Beispiel"; datum = "2026-02-03"; betrag = "17,90" }
    )
PS> $zeilen | Export-Csv beispiel.csv -NoTypeInformation -Encoding utf8
```

WARNUNG – Sicherheitsrisiko

Ein schneller Zugriff nur auf die Spalte `name` einer **echten** Kundendatei ist **keine verlässliche Anonymisierung**: Andere Spalten (E-Mail, Adresse, Freitextfelder) bleiben komplett unangetastet, und schon die Kombination aus Datum, Betrag und sonstigen Angaben kann eine Person wieder eindeutig identifizierbar machen. Für eine KI-Anfrage brauchst du in aller Regel ohnehin keine echten Daten, sondern nur deren **Struktur** – erzeuge sie stattdessen selbst, wie oben. Musst du wirklich mit einer echten Datei arbeiten, lege vorab eine Liste der erlaubten Spalten fest (Allowlist) und prüfe Freitextfelder gesondert von Hand.

WARNUNG – Sicherheitsrisiko

Für personenbezogene Daten in Deutschland und der EU gelten die Datenschutz-Grundverordnung (DSGVO) und das Bundesdatenschutzgesetz. Wer solche Daten **beruflich** an einen KI-Dienst schickt, braucht dafür in der Regel eine Rechtsgrundlage und meist einen Auftragsverarbeitungsvertrag. Frage vorher die zuständige Stelle in deiner Organisation. Der sicherste Weg bleibt: echte Datensätze erst gar nicht in externe Chats kopieren.

6.9 Wann du KI-Code NICHT vertrauen darfst

KI-Code ist ein Entwurf, nie ein Urteil. In diesen Fällen brauchst du besondere Vorsicht bis hin zu **fachkundiger menschlicher Prüfung**:

Bereich	Warum kritisch
Kryptografie und Verschlüsselung	Kleine Fehler machen alles wertlos; nutze nur bewährte Bibliotheken

Bereich	Warum kritisch
Anmeldung, Sitzungen, Rechtevergabe	Fehler öffnen Türen für Angreifer
Zahlungen und Finanzdaten	Hoher Schaden, rechtliche Pflichten
Medizin, Sicherheit, Verkehr, Industrie	Gefahr für Menschen
Betrieb im Internet	Ständige Angriffe auf offene Dienste
Systemadministration mit Administratorrechten	Ein falscher Befehl beschädigt das System
Datenlöschung, Migration, Backups	Fehler sind oft nicht umkehrbar
Rechtliche, steuerliche Berechnungen	Falsche Annahmen mit realen Folgen

Faustregel: **Je höher Schaden und Unumkehrbarkeit, desto weniger Vertrauen und desto mehr menschliche Prüfung.** Für ein Spielprojekt reicht „läuft“. Für alles, was Menschen, Geld oder fremde Daten betrifft, reicht es nicht.

Übungen

Übung

1. Durchsuche eines deiner Projekte mit Beispiel 6.2 nach Geheimnissen. Gibt es Treffer? Wie würdest du sie beheben?
2. Lass dir von der KI drei gefährliche Terminalbefehle nennen und jeweils erklären, was sie anrichten – führe sie **nicht** aus.
3. Schreibe das unsichere Beispiel 6.5 in einer Datei und teste in `$env:TEMP`, was bei der Eingabe `localhost & echo GEHACKT` passiert (verwende `echo` statt `del`!). Teste danach die sichere Variante.
4. Erzeuge mit `pruefe.ps1` (Beispiel 6.8) eine Prüfroutine für ein eigenes Projekt und binde sie in deinen Ablauf vor jedem Commit ein.
5. Entscheide für fünf Projektideen, ob du KI-Code ungeprüft, geprüft oder nur mit fachkundiger Prüfung einsetzen würdest.

Lösungshinweis

Zu 3: Bei der unsicheren Variante gibt die Shell nach dem ersten Befehl auch `GEHACKT` aus – der eingeschleuste Befehl wurde ausgeführt. Die sichere Variante behandelt die Eingabe als Hostnamen und meldet, dass der Host nicht gefunden wurde.

Zusammenfassung

Zusammenfassung

- Keys gehören in Umgebungsvariablen, nie in Code, Chat oder Repository; bei Leck sofort widerrufen.
- Prüfe jeden vorgeschlagenen Befehl mit den drei Fragen; starte Agenten nur in Projektordnern.
- Typische Schwachstellen: Befehls-, SQL- und Pfad-Einschleusung sowie XSS; offene Server und selbstgebaute Anmeldung.
- Eine feste Prüfroutine (Tests, Linter, Geheimnis-Suche, `git diff`) gehört vor jeden Commit.
- Prüfe Pakete vor der Installation; bevorzuge die Standardbibliothek.
- Lizenz und Datenschutz: Sobald es beruflich oder kommerziell wird, kläre Rechtsfragen fachkundig.

- Bei Krypto, Anmeldung, Zahlungen und sicherheitskritischen Systemen gilt: nie ungeprüft vertrauen.

Kapitel 7: Typische Fallstricke und Troubleshooting

Lernziele

- Du erkennst Halluzinationen und überprüfst Behauptungen der KI.
- Du gehst mit veraltetem Code und Versionskonflikten um.
- Du löst Abhängigkeitsprobleme systematisch.
- Du gehst konstruktiv mit „Es funktioniert, aber ich verstehe es nicht“ um.

7.1 Halluzinationen

Eine **Halluzination** ist eine überzeugend klingende, aber falsche Aussage der KI. Beim Programmieren treten sie meist in diesen Formen auf:

Form	Beispiel
Erfundene Funktion	`datei.lese_alles_sicher()` – gibt es nicht
Erfundene Option	`Select-String -NurTreffer` – existiert nicht
Erfundenes Paket	`pip install superparser-pro` – unbekannt oder gefährlich
Falsche Version	Funktion existiert erst in einer neueren Version
Erfundene Quelle	Verweis auf eine Dokumentationsseite, die es nie gab
Falsche Selbstauskunft	„Ich habe den Code getestet“ – ohne Ausführung

So überprüfst du Behauptungen

1. **Existiert der Befehl?** Nutze die eingebaute Hilfe.
2. **Existiert die Funktion?** Frage die Python-Hilfe ab.
3. **Stimmt die Option?** Schau in die Hilfe (`Get-Help`).
4. **Läuft es wirklich?** Nur Ausführen beweist es.

Beispiel 7.1: Behauptungen schnell nachprüfen

```
PS> Get-Command jq                              # gibt es das Programm?
PS> Get-Help Select-String -Full | more         # PowerShell-Hilfe (q beendet)
PS> git --help                                  # Kurzhilfe zu einem Programm
PS> py -c "import pathlib; help(pathlib.Path.read_text)"
PS> py -c "import json; print(dir(json))"       # welche Namen gibt es?
PS> winget search jq                            # Paket und Version in winget
```

Gegenmaßnahmen im Prompt

- „Wenn du dir bei einer Funktion oder Option nicht sicher bist, sage es ausdrücklich.“
- „Nenne die Python-Version, ab der die Funktion existiert.“
- „Erfinde keine Pakete. Wenn die Standardbibliothek reicht, nutze sie.“
- „Verweise auf offizielle Dokumentation und nenne das Stichwort, nach dem ich suchen kann.“

> **Gut zu wissen**
>
> Frage die KI nie: „Bist du sicher?“ als einzigen Test. Manche Modelle rudern dann auch bei richtigen Antworten zurück. Besser: „Wie kann ich das selbst überprüfen?“ Das liefert dir einen Testbefehl.

7.2 Veralteter Code und Versionskonflikte

Modelle lernen aus Daten bis zu einem Stichtag. Bibliotheken ändern sich danach weiter. Die Folge: Die KI schlägt Aufrufe vor, die in der neuen Version geändert oder entfernt wurden, oder umgekehrt Funktionen einer neueren Version, die bei dir noch nicht existieren.

Typische Symptome

- `AttributeError: module 'x' has no attribute 'y'`
- `DeprecationWarning` oder `FutureWarning` in der Ausgabe
- Beispiele aus Tutorials, die nicht mehr laufen
- Befehle mit Optionen, die deine Programmversion nicht kennt

Vorgehen

Beispiel 7.2: Versionen feststellen

```
PS> py --version
PS> pip show flask | Select-Object -First 3   # Name, Version
PS> pip list --outdated                       # veraltete Pakete (in der venv)
PS> node --version                            # falls JavaScript-Tooling im Spiel ist
```

Gib der KI die **tatsächlichen Versionen** und bitte um Code für genau diese:

Beispiel 7.3: Versionsbewusster Prompt

```
Ich nutze Python 3.11 und Flask 3.0.2 unter Windows 11. Dein Code verwendet
eine Funktion, die dort offenbar nicht existiert (Fehlermeldung unten).
Prüfe, ob sie in meiner Version existiert; falls nicht, nenne die korrekte
Alternative für genau diese Versionen und verweise auf den Dokumentationsabschnitt.
```

> **TIPP – Profi-Trick**
>
> Wenn die KI Webzugriff hat, bitte sie ausdrücklich, die aktuelle offizielle Dokumentation nachzuschlagen. Ohne Webzugriff: Kopiere den relevanten Abschnitt der Dokumentation selbst in den Chat.

Pin: Versionen festlegen

Damit ein Projekt auch in Monaten noch läuft, **friere** du die Versionen ein (siehe Beispiel 2.7). So sind Fehler später reproduzierbar, und Updates machst du bewusst, nicht aus Versehen.

7.3 Abhängigkeitskonflikte

Ein **Abhängigkeitskonflikt** entsteht, wenn zwei Pakete verschiedene Versionen eines dritten verlangen oder wenn Pakete mit deiner Python-Version nicht zusammenpassen.

Häufige Fehlermeldungen

Meldung	Bedeutung und Lösung
`Die Ausführung von Skripts ist deaktiviert`	Ausführungsrichtlinie anpassen (Kapitel 2.3)
`ModuleNotFoundError: No module named 'x'`	Paket fehlt oder falsche Umgebung aktiv. `Get-Command python` prüfen
`ERROR: Cannot install a and b because these package versions have conflicting dependencies`	Versionskonflikt. Eine Version festlegen oder Paket weglassen
`Microsoft Visual C++ 14.0 or greater is required`	Compiler fehlt: Visual Studio Build Tools installieren oder Paketversion mit fertigem Wheel wählen
`No matching distribution found`	Paketname falsch oder Python zu alt/neu
`Access is denied` bei `pip install`	Nicht als Administrator installieren; venv nutzen

Systematische Fehlersuche

Beispiel 7.4: Die richtige Umgebung prüfen

```
PS> Get-Command python, pip | Select-Object Source   # Aus der venv? Pfad sollte auf .venv zeigen
PS> py -c "import sys; print(sys.prefix)"
PS> py -m pip list                # Was ist hier installiert?
PS> py -m pip check               # Meldet kaputte Abhängigkeiten
```

Wenn gar nichts mehr hilft, ist der Neustart die schnellste Lösung:

Beispiel 7.5: Virtuelle Umgebung sauber neu aufbauen

```
PS> deactivate
PS> Remove-Item -Recurse -Force .venv
PS> py -m venv .venv
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> pip install -r requirements.txt
```

Das kostet eine Minute und löst erstaunlich viele „unerklärliche“ Probleme. Weil die venv nur Wegwerf-Material enthält (und in `.gitignore` steht), ist das gefahrlos.

> **TIPP – Profi-Trick**
>
> Aktiviere bei Problemen zuerst **den Verdacht auf die Umgebung**: Falsche venv aktiv? Falscher Python? Paket in einer anderen Umgebung installiert? Das ist bei Anfängern die häufigste Ursache.

7.4 „Es funktioniert, aber ich verstehe es nicht“

Das ist die typische Falle des Vibe Codings: Der Code läuft, und du hast keine Ahnung, warum. Solange nichts schiefgeht, ist das kein Problem. Sobald etwas kaputtgeht, schon.

Warum das gefährlich ist

- Du kannst Fehler nicht eingrenzen.
- Du merkst nicht, wenn die KI etwas Riskantes eingebaut hat.
- Jede Erweiterung wird zum Glücksspiel.

Die Verständnis-Leiter

Du musst nicht alles verstehen, aber du solltest **stufenweise** verstehen:

1. **Zweck:** Was tut das Programm als Ganzes? (Ein Satz.)
2. **Aufbau:** Welche Teile (Funktionen, Dateien) gibt es und wofür?
3. **Ablauf:** In welcher Reihenfolge geschieht was?
4. **Details:** Was tut diese Zeile? Warum so?

Beginne oben. Meist genügt Stufe 2 bis 3, um Fehler gezielt zu suchen.

Beispiel 7.6: Prompt für Verständnis in Stufen

```
Erkläre mir dieses Programm in vier Stufen, jeweils kurz:
1. Zweck in einem Satz
2. Aufbau: Liste der Funktionen mit je einem Satz
3. Ablauf: Was passiert vom Start bis zur Ausgabe, in nummerierten Schritten
4. Drei Stellen, an denen es am ehesten schiefgehen kann
Ich bin Anfänger und kenne nur Terminal-Grundlagen.
```

Techniken, die Verständnis erzeugen

- **Kommentieren lassen:** „Füge verständliche Kommentare ein, die erklären, warum, nicht nur was."
- **Kleine Änderung selbst machen:** Ändere eine Zahl oder einen Text und beobachte die Folge.
- **Zeile auskommentieren:** Mit `#` in Python lässt du eine Zeile weg und siehst, was fehlt.
- **Spielzeug-Version:** Lass dir die Kernidee in fünf Zeilen als Mini-Beispiel zeigen.
- **Erklären lernen:** Erkläre das Programm der KI in eigenen Worten und lass dich korrigieren. Das ist eine der wirksamsten Lernmethoden überhaupt.
- **Debugger oder Diagnose-Ausgaben:** Schau zu, wie sich Werte beim Ablauf ändern (Beispiel 3.11).

> **Gut zu wissen**
>
> Ein Verständnis-Ziel für Anfänger: Du kannst für jedes deiner Programme in zwei Minuten jemandem erklären, **was es tut, aus welchen Teilen es besteht und wo du bei einem Fehler zuerst suchen würdest.** Das reicht für den Anfang weit.

7.5 Weitere Fallstricke

Die KI ändert zu viel

Symptom: Du bittest um eine Kleinigkeit und bekommst eine umgebaute Datei. Gegenmittel: „Ändere nur X, sonst nichts", „Zeige nur das Diff", Prüfung mit `git diff`, kleinere Schritte.

Die Endlosschleife

Symptom: Fix A bricht B, Fix B bricht A. Gegenmittel: Zurück zum letzten guten Stand (`git restore .`), Ursache **erklären** lassen, Problem isolieren (Abschnitt 3.7).

Übermäßige Komplexität

Die KI baut gern große Strukturen: Klassen, Frameworks, Konfigurationsdateien für ein 30-Zeilen-Problem. Bitte um „die einfachste Lösung, die funktioniert" und „eine Datei, nur Standardbibliothek".

Zu viel Vertrauen in grüne Tests

Die KI schreibt manchmal Tests, die ihr eigenes Verhalten bestätigen, statt das Soll zu prüfen. Prüfe die erwarteten Werte selbst nach: Stimmt „erwartet“ mit dem überein, was **du** möchtest?

Stilles Scheitern

Code, der Fehler verschluckt (`except: pass`), meldet nie ein Problem. Suche solche Stellen gezielt:

Beispiel 7.7: Verdächtige Fehlerbehandlung aufspüren

```
PS> rg -n 'except\s*:|except Exception:\s*$|pass$' --glob '*.py' .
PS> rg -n -e 'SilentlyContinue' -e '2>\$null' --glob '*.ps1' .
```

Nicht jeder Treffer ist falsch, aber jeder ist eine Frage wert: „Warum wird dieser Fehler ignoriert?“

Zeichensatz und Zeilenenden

Unter Windows gibt es zwei klassische Stolpersteine. Erstens: **Windows PowerShell 5.1** schreibt Dateien mit `>` und `Out-File` standardmäßig in UTF-16; das verwirrt Python, `pip` und viele andere Programme. Zweitens: Windows beendet Zeilen mit zwei Zeichen (CRLF), Linux mit einem (LF). Das stört, wenn ein Skript später in WSL oder auf einem Linux-Server läuft (`bad interpreter`), und führt in Git zu Meldungen wie „LF will be replaced by CRLF“.

Beispiel 7.8: Zeichensatz und Zeilenenden prüfen und reparieren

```
PS> "Test äöü" | Set-Content test.txt -Encoding utf8          # sicher: UTF-8
PS> Select-String -Path skript.py -Pattern "[^\x00-\x7F]"      # Nicht-ASCII-Zeichen finden
PS> $t = Get-Content skript.sh -Raw
PS> $t -replace "`r`n", "`n" | Set-Content skript.sh -NoNewline  # CRLF -> LF
PS> git config --global core.autocrlf true                     # Git kümmert sich darum
```

Typische Symptome sind Umlaute als Kauderwelsch, „kann nicht gelesen werden“-Fehler und unsichtbare Zeichen am Zeilenende. Speichere Quelltext im Editor immer als **UTF-8**; VS Code zeigt Codierung und Zeilenende unten rechts an und lässt sich dort umstellen.

Einrückung in Python

Mischung aus Tabs und Leerzeichen erzeugt `IndentationError`. Nutze im Editor durchgehend vier Leerzeichen; lass die KI den Code neu formatieren oder nutze `ruff format`.

7.6 Ein Fahrplan für jedes Problem

1. **Ruhig bleiben, Meldung lesen** – vollständig.
2. **Reproduzieren:** Tritt es jedes Mal auf? Mit welchem Befehl?
3. **Eingrenzen:** Was hat sich seit dem letzten funktionierenden Stand geändert? (`git diff`, `git stash`)
4. **Umgebung prüfen:** Versionen, venv, Rechte, Pfade.
5. **Erklären lassen,** dann erst Fix vorschlagen lassen.
6. **Ein Fix auf einmal,** sofort testen.
7. **Dokumentieren:** Ursache und Lösung ins Lerntagebuch.

> **TIPP – Profi-Trick**

`git bisect` findet automatisch den Commit, der einen Fehler eingeführt hat. Dafür brauchst du viele kleine Commits – ein weiterer Grund für die Regel „Ein Schritt = ein Commit“. Frage die KI nach einer Anleitung, sobald du es brauchst.

Übungen

Übung

1. Bitte die KI um eine Funktion, die es nicht gibt (erfinde einen Namen). Wie reagiert sie? Prüfe mit `help()` oder `dir()`.
2. Erzeuge absichtlich einen `ModuleNotFoundError` (Import eines nicht installierten Pakets) und löse ihn mit venv und `pip install`.
3. Baue die venv eines Projekts nach Beispiel 7.5 neu auf.
4. Nimm ein Programm, das du nicht verstehst, und arbeite die Verständnis-Leiter (Beispiel 7.6) durch. Erkläre es dann der KI in eigenen Worten.
5. Suche mit Beispiel 7.7 in einem Projekt nach stillem Scheitern und bewerte jeden Treffer.

Lösungshinweis

Zu 2: `py -c "import notexistingpkg"` erzeugt `ModuleNotFoundError`. Mit aktivierter venv installierst du ein echtes Paket mit `pip install paketname`. Wichtig: Prüfe vorher mit `Get-Command python`, dass die venv aktiv ist.

Zusammenfassung

Zusammenfassung

- Halluzinationen erkennst du nur durch Gegenprüfung: `command -v`, `man`, `help()`, Ausführen.
- Gib der KI die tatsächlichen Versionen und friere Abhängigkeiten ein.
- Bei Abhängigkeitsproblemen zuerst die Umgebung prüfen; eine frische venv löst vieles.
- Verständnis wächst stufenweise: Zweck, Aufbau, Ablauf, Details.
- Typische Probleme: zu große Änderungen, Endlosschleifen, Überkomplexität, stilles Scheitern, Zeilenenden.
- Der Fahrplan: lesen, reproduzieren, eingrenzen, Umgebung prüfen, erklären lassen, ein Fix, dokumentieren.

Kapitel 8: Nächste Schritte

Lernziele

- Du planst deinen persönlichen Lernpfad vom Vibe Coder zum sicheren Entwickler.
- Du kennst gute Quellen und Gemeinschaften zum Weiterlernen.
- Du wählst Folgeprojekte passend zu deinem Interesse.
- Du hast eine Checkliste für deinen Alltag als Vibe Coder.

8.1 Wo stehst du jetzt?

Wenn du dieses Handbuch durchgearbeitet hast, kannst du:

- eine Umgebung mit Editor, Python, Git und KI-Assistent einrichten,
- Aufgaben in kleine Schritte zerlegen und mit der KI umsetzen,
- Fehlermeldungen auswerten und Code reviewen lassen,
- drei Arten von Projekten bauen (CLI, Web, Automatisierung),
- Risiken erkennen: Keys, Befehle, Pakete, Daten.

Das ist ein solides Fundament. Der Unterschied zu einem erfahrenen Entwickler liegt vor allem im **Urteilsvermögen**: Welche Lösung ist einfach, robust und wartbar? Dieses Gefühl entsteht durch Praxis und durch immer tieferes Verstehen.

8.2 Der Lernpfad in vier Stufen

Stufe	Ziel	Typische Aktivitäten	Zeitrahmen (Richtwert)
1. Vibe Coder	Funktionierende Werkzeuge bauen	Dieses Handbuch, kleine Projekte	2 bis 4 Wochen
2. Verstehender Nutzer	Code lesen und anpassen	Eigene Änderungen, Tests, Debugging	2 bis 3 Monate
3. Selbstständiger Entwickler	Programme selbst entwerfen	Python-Grundlagen systematisch, Datenstrukturen, Module	6 bis 12 Monate
4. Fortgeschritten	Größere Systeme, Qualität	Architektur, Datenbanken, Sicherheit, Teamarbeit	fortlaufend

Die Zeitangaben sind grobe Richtwerte und hängen stark von deiner Übungszeit ab. Wichtiger als die Dauer ist die Regelmäßigkeit: lieber 30 Minuten täglich als ein Marathon pro Monat.

Was du auf Stufe 2 als Nächstes lernen solltest

Diese Grundlagen machen dich unabhängiger von der KI und deine Anfragen präziser:

1. **Variablen, Datentypen** (Text, Zahlen, Listen, Wörterbücher)
2. **Kontrollfluss:** `if`, `for`, `while`
3. **Funktionen und Module**
4. **Dateien lesen und schreiben**
5. **Fehlerbehandlung** (`try`/`except`)

6. **Tests schreiben** und ausführen
7. **Git im Alltag:** Branches, Merge, Remote-Repositories
8. **Debuggen:** Diagnose-Ausgaben, Debugger

Ein einfacher Lernrhythmus

Beispiel 8.1: Wochenplan für Stufe 2

```
- Montag, 30 min: Ein Konzept lernen (z. B. Schleifen) – KI erklärt mit Alltagsbeispiel
- Mittwoch, 45 min: Eine Mini-Aufgabe OHNE KI-Code lösen, dann mit KI vergleichen
- Freitag, 45 min: Am eigenen Projekt eine kleine Erweiterung bauen
- Sonntag, 10 min: Lerntagebuch schreiben und Wochenrückblick
```

> **TIPP – Profi-Trick**
>
> **Die 70/30-Regel zum Lernen:** Versuche eine Aufgabe zuerst selbst, mindestens ein paar Minuten lang, bevor du die KI fragst. Dann lernst du aus der Differenz zwischen deinem Versuch und ihrer Lösung. Wer sofort fragt, lernt deutlich weniger. Lass dir außerdem Hinweise statt Komplettlösungen geben: „Gib mir nur einen Tipp, nicht die Lösung."

8.3 Die KI als Lehrer einsetzen

Du kannst die KI gezielt als geduldigen Nachhilfelehrer verwenden:

Beispiel 8.2: Der KI-Tutor-Prompt

```
Du bist mein Python-Tutor. Ich bin Windows-Anwender mit Terminal-Erfahrung,
aber Programmieranfänger. Gib mir ein Mini-Programmierproblem zum Thema
"Schleifen". Löse es NICHT, sondern warte auf meinen Lösungsversuch.
Gib mir danach Rückmeldung: was gut ist, was fehlt, und einen Hinweis statt
der fertigen Lösung. Steigere die Schwierigkeit langsam.
```

Weitere Lernformate:

- **Code-Lesen:** „Zeige mir ein 20-Zeilen-Programm und frage mich, was es tut."
- **Fehlersuche:** „Baue einen versteckten Fehler ein, und ich suche ihn."
- **Erklären-lassen:** „Prüfe mein Verständnis mit fünf Fragen zu diesem Kapitel."
- **Projektbegleitung:** „Begleite mich beim Bau von X, aber schreibe nur den Code, den ich ausdrücklich verlange."

8.4 Folgeprojekte nach Interesse

Wähle Projekte, die du **wirklich nutzen** würdest. Das motiviert am meisten.

Interesse	Projektidee	Neu zu lernen
Systemadministration	Backup-Skript mit Rotation und Bericht per Mail	PowerShell, robocopy, Fehlerbehandlung, Aufgabenplanung
Heimnetz	Dashboard mit Geräten, Speicher, Temperatur	Systeminfos auslesen, kleine Weboberfläche
Daten	CSV-Auswertung mit Diagrammen	Python-Bibliotheken für Daten und Grafik

Interesse	Projektidee	Neu zu lernen
Web	Persönliche Webseite oder Blog (statisch)	HTML, CSS, Hosting, Domain
Alltag	Haushaltsbuch im Terminal	Dateiformate, Datenbank (SQLite), Tests
Medien	Bildersortierer anhand der Aufnahmedaten	Metadaten, Dateioperationen
KI	Eigenes Skript, das einen KI-Dienst über die API anspricht	HTTP, JSON, Umgang mit Keys (Kapitel 6)

Ein Projekt mit KI-API

Der nächste logische Schritt: Dein Programm ruft selbst eine KI auf. Dafür liest du den Key aus der Umgebung (Beispiel 6.1) und sendest eine Anfrage über HTTP. Die genaue Anleitung steht in der Dokumentation deines Anbieters; frage dort nach dem aktuellen Beispiel, weil sich Modellnamen und Schnittstellen ändern. Beachte Kosten (Ausgabenlimit!), Datenschutz (Kapitel 6.8) und Fehlerbehandlung bei Netzwerkproblemen.

> **WARNUNG – Sicherheitsrisiko**
>
> Programme, die automatisch KI-Dienste aufrufen, können durch Fehler (Endlosschleifen) hohe Kosten verursachen. Setze ein Ausgabenlimit im Konto, begrenze Wiederholungen im Code und teste zuerst mit wenigen kleinen Anfragen.

8.5 Ressourcen zum Weiterlernen

Die Qualität und Verfügbarkeit von Angeboten ändert sich; prüfe Aktualität und Qualität selbst. Bewährte Anlaufstellen:

Offizielle Dokumentation (zuverlässigste Quelle)

- **Python:** docs.python.org, vor allem das offizielle Tutorial (auch auf Deutsch verfügbar)
- **Git:** git-scm.com, das Buch „Pro Git“ ist dort kostenlos und auf Deutsch lesbar
- **MDN Web Docs:** developer.mozilla.org, die beste Referenz für HTML, CSS, JavaScript
- **PowerShell:** `Get-Help` im Terminal und die Dokumentation auf learn.microsoft.com
- **Flask:** flask.palletsprojects.com
- **Dokumentation deines KI-Werkzeugs:** Funktionen und Befehle ändern sich schnell; lies sie regelmäßig

Lernplattformen und Kurse

- Das kostenlose Programmierangebot **freeCodeCamp**
- Das Einsteigerprogramm **CS50** der Harvard University (online frei zugänglich)
- Interaktive Python-Kurse und Übungsplattformen (zum Beispiel Exercism)
- **VHS, IHK und Weiterbildungsträger** bieten Kurse für Einsteiger in Präsenz und online

Bücher

- „Automate the Boring Stuff with Python“ von Al Sweigart: genau für Menschen, die Alltagsaufgaben automatisieren möchten (online frei lesbar, auch auf Deutsch erschienen)
- „Pro Git“ von Scott Chacon und Ben Straub
- „PowerShell in a Month of Lunches“ (falls du die Windows-Kommandozeile vertiefen willst)

Gemeinschaften

- **Foren und Q&A-Seiten** (zum Beispiel Stack Overflow und deutschsprachige IT-Foren)
- **Subreddits und Discord-Server** zu Python, Windows und KI-gestütztem Programmieren
- **Lokale Gruppen:** Usergroups, Treffs des Chaos Computer Clubs, Meetups, Repair-Cafés mit Programmier-Tisch
- **Open-Source-Projekte:** Beiträge zu Dokumentation und kleinen Fehlern sind ein hervorragender Einstieg

> **TIPP – Profi-Trick**
>
> Wenn du in einem Forum fragst, nutze das Muster aus Beispiel 3.9: Ziel, Befehl, vollständige Fehlermeldung, Umgebung, und schreibe, was du schon versucht hast. Damit bekommst du deutlich bessere Antworten, und du zeigst, dass du dich bemüht hast.

8.6 Berufliche Perspektiven realistisch einschätzen

Vibe Coding ist ein starkes Werkzeug für private Projekte, Prototypen und Automatisierung am Arbeitsplatz. Als alleinige Grundlage für Software, die Kunden bezahlen, ist es dünn. Wer beruflich einsteigen möchte, braucht zusätzlich:

- **Grundlagenwissen** in einer Programmiersprache, Datenstrukturen und Entwurfsprinzipien,
- **Fähigkeit zur Fehlersuche ohne KI,**
- **Sicherheitsbewusstsein** (Kapitel 6),
- **Teamwerkzeuge** (Git-Workflows, Code-Reviews, Tickets),
- **Nachweise** in Form von Projekten, die du zeigen und erklären kannst.

In der IT-Systemintegration, in der Administration und im Support sind Skripte und Automatisierung schon heute ein großer Vorteil. Auch hier gilt: Wer die Werkzeuge versteht und Verantwortung für Qualität und Sicherheit übernimmt, ist wertvoller als jemand, der nur Code erzeugen lässt.

> **Gut zu wissen**
>
> Ein Portfolio muss nicht groß sein: drei saubere Projekte mit README, Tests und klarer Beschreibung, was du selbst gemacht hast und was die KI beigetragen hat, wirken überzeugender als zehn halbfertige. Ehrlichkeit über KI-Unterstützung wird in der Branche zunehmend erwartet.

8.7 Deine Alltags-Checkliste

Drucke sie aus oder lege sie in `$HOME\vibe\notizen\checkliste.md` ab:

Beispiel 8.3: Checkliste für jedes Projekt

```
## Vor dem Start
- [ ] Ziel in einem Satz, MVP und Plan mit Testbefehlen
- [ ] Projektordner, Git-Repository, venv, .gitignore
- [ ] PROJEKT.md mit Rahmenbedingungen

## Während der Arbeit
- [ ] Kleiner Schritt, ein Test, ein Commit
- [ ] git diff gelesen
- [ ] Keine Geheimnisse im Chat oder Code
- [ ] Riskante Befehle erklärt und geprüft

## Vor dem Einsatz
- [ ] Normal-, Fehler- und Randfälle getestet
- [ ] Trockenlauf bei Programmen, die Daten verändern
- [ ] Backup vorhanden
- [ ] Abhängigkeiten geprüft, requirements.txt aktuell
- [ ] README geschrieben

## Danach
- [ ] Lerntagebuch ergänzt
- [ ] Offene Fragen notiert
```

Schlusswort

Vibe Coding senkt die Einstiegshürde, nimmt dir aber nicht die Verantwortung ab. Du hast jetzt die Werkzeuge: einen Arbeitsablauf, der kleine Schritte belohnt, ein Sicherheitsnetz mit Git, Prüfroutinen gegen die häufigsten Fehler und die Haltung, KI-Code als Entwurf zu behandeln. Baue Dinge, die du wirklich brauchst. Mach Fehler in sicherer Umgebung. Frage dich bei jedem Programm: „Verstehe ich, was es tut?“, und lerne mit jedem Projekt ein Stück mehr.

Viel Freude und Erfolg beim Bauen.

Übungen

Übung

1. Lege deinen persönlichen Lernplan für die nächsten vier Wochen an (Beispiel 8.1).
2. Wähle ein Folgeprojekt aus Tabelle 8.4 und schreibe Plan und `PROJEKT.md`.
3. Starte eine Tutor-Sitzung nach Beispiel 8.2 zu einem Thema, das du noch nicht verstehst.
4. Erstelle die Checkliste aus Beispiel 8.3 und wende sie auf eines deiner bisherigen Projekte an.
5. Schreibe eine README für dein bestes Projekt, die auch jemand ohne Vorwissen versteht.

Zusammenfassung

Zusammenfassung

- Dein Lernpfad führt vom Vibe Coder über den verstehenden Nutzer zum selbstständigen Entwickler.
- Versuche Aufgaben zuerst selbst; nutze die KI als Tutor, nicht nur als Lieferant.
- Wähle Folgeprojekte, die du wirklich nutzt; steigere Schwierigkeit und Verantwortung langsam.
- Offizielle Dokumentation, freie Kurse, Bücher und Gemeinschaften helfen beim Weiterlernen.
- Beruflich brauchst du zusätzlich Grundlagenwissen, Sicherheitsbewusstsein und herzeigbare Projekte.

- Eine feste Checkliste macht gute Praxis zur Gewohnheit.

Glossar

Abhängigkeit – Fremdes Programmpaket, das dein Projekt zum Laufen braucht.

Agent (KI-Agent) – KI-Werkzeug, das selbstständig Schritte ausführt, etwa Dateien lesen, ändern und Befehle ausführen.

API – Schnittstelle, über die Programme miteinander oder mit Diensten sprechen.

API-Key – Geheimer Zugangsschlüssel zu einem Dienst; wer ihn kennt, nutzt den Dienst auf deine Rechnung.

Argument – Wert, den du einem Programm oder einer Funktion beim Aufruf mitgibst.

Aufgabenplanung – Windows-Dienst zum zeitgesteuerten Ausführen von Programmen (englisch: Task Scheduler).

Ausführungsrichtlinie – Windows-Einstellung, die festlegt, welche PowerShell-Skripte laufen dürfen.

Backend – Teil einer Anwendung, der auf dem Server läuft (Daten, Logik).

Branch – Eigener Entwicklungszweig in Git.

Code-Review – Kritische Durchsicht von Code auf Fehler, Sicherheit und Lesbarkeit.

Commit – Gespeicherter Schnappschuss des Projekts in Git.

CRLF – Zeilenende unter Windows (zwei Zeichen); Linux nutzt nur LF.

CSS – Sprache für das Aussehen von Webseiten.

Debugging – Fehler finden und beheben.

Diff – Gegenüberstellung von Änderungen (neu mit +, entfernt mit -).

DSGVO – Datenschutz-Grundverordnung der EU.

Exit-Code – Zahl, die ein Programm beim Beenden zurückgibt; 0 bedeutet Erfolg.

Flask – Kleines Python-Framework für Webanwendungen.

Framework – Fertiges Gerüst, das viel Grundarbeit abnimmt und Regeln vorgibt.

Funktion – Benannter, wiederverwendbarer Programmbaustein.

Git – Werkzeug zur Versionskontrolle.

Halluzination – Überzeugend klingende, aber falsche Ausgabe einer KI.

Hash – Digitaler Fingerabdruck eines Dateiinhalts (zum Beispiel SHA-256).

HTML – Auszeichnungssprache für den Inhalt von Webseiten.

IDE – Entwicklungsumgebung: Editor mit Zusatzfunktionen für Programmierer.

Iteration – Wiederholter Durchlauf aus Ändern, Testen und Verbessern.

JavaScript – Programmiersprache für Verhalten in Webseiten.

JSON – Textformat für strukturierte Daten.

Kontextfenster – Textmenge, die ein KI-Modell pro Gespräch gleichzeitig berücksichtigen kann.

Linter – Werkzeug, das Code automatisch auf typische Fehler und Stilprobleme prüft.

LLM – Large Language Model, großes Sprachmodell.

localStorage – Speicher im Browser für kleine Datenmengen.

MVP – Minimal funktionsfähige Version eines Projekts.

Neovim – Terminal-Editor mit Modi und hoher Anpassbarkeit.

Path Traversal – Angriff, bei dem Eingaben wie ../ aus einem erlaubten Ordner ausbrechen.

PowerShell – Windows-Shell und Skriptsprache mit Befehlen wie Get-ChildItem.

Prompt – Anweisung oder Frage an eine KI.

Repository – Projektordner samt Git-Verlauf.

Schleife – Programmstruktur, die etwas mehrfach wiederholt.

Shell – Programm, das Terminalbefehle entgegennimmt (zum Beispiel Bash).

SQL – Sprache für Datenbankabfragen.

SQL-Injection – Angriff, bei dem Eingaben Datenbankbefehle einschleusen.

Standardbibliothek – Sammlung von Modulen, die mit der Programmiersprache mitgeliefert wird.

Test – Kleines Programm, das prüft, ob Code das Richtige tut.

Token – Text-Baustein, in dem ein Sprachmodell rechnet und abrechnet.

Traceback – Python-Fehlerbericht mit dem Weg zur Fehlerstelle.

Trockenlauf – Probelauf, der nur anzeigt, was passieren würde.

Umgebungsvariable – Benannter Wert, den die Umgebung an Programme weitergibt.

UTF-8 – Weit verbreitete Zeichencodierung, die auch Umlaute zuverlässig speichert.

venv – Virtuelle Python-Umgebung mit eigenen Paketen pro Projekt.

Versionskontrolle – Verfahren, um Änderungen nachvollziehbar zu speichern und zurückzunehmen.

Vibe Coding – Software durch Beschreibung in natürlicher Sprache mit einer KI entwickeln.

Windows Sandbox – Wegwerf-Windows in einem Fenster, das beim Schließen alles löscht.

winget – Paketmanager von Windows zum Installieren von Programmen per Kommandozeile.

WSL – Windows-Subsystem für Linux; startet eine Linux-Umgebung in Windows.

XSS – Cross-Site-Scripting: Einschleusen von Schadcode über Nutzertexte in Webseiten.

Verzeichnis der Codebeispiele

Beispiel 1.1: Eine erste Anfrage an die KI … Seite 11
Beispiel 1.2: Typische Antwort – ein Hallo-Programm … Seite 11
Beispiel 1.3: Programm starten … Seite 11
Beispiel 2.1: Vorhandene Werkzeuge prüfen … Seite 13
Beispiel 2.2: Grundpakete mit winget installieren … Seite 14
Beispiel 2.3: Zusatzwerkzeuge installieren … Seite 14
Beispiel 2.4: Arbeitsordner anlegen … Seite 15
Beispiel 2.5: Virtuelle Umgebung anlegen und aktivieren … Seite 16
Beispiel 2.6: Ausführungsrichtlinie für dein Konto lockern (einmalig) … Seite 16
Beispiel 2.7: Abhängigkeiten festhalten und wiederherstellen … Seite 16
Beispiel 2.8: Ordner in VS Code öffnen … Seite 17
Beispiel 2.9: Neovim starten und das Tutorial öffnen … Seite 18
Beispiel 2.10: Typischer Ablauf mit einem Terminal-Agenten … Seite 18
Beispiel 2.11: Verlaufsdatei von PowerShell finden … Seite 19
Beispiel 2.12: API-Key sicher ablegen … Seite 19
Beispiel 2.13: Key in die Sitzung laden und prüfen … Seite 19
Beispiel 2.14: Git konfigurieren … Seite 20
Beispiel 2.15: Erstes Repository mit erstem Commit … Seite 20
Beispiel 2.16: Eine sinnvolle .gitignore für Python-Projekte … Seite 21
Beispiel 2.17: Skript "neues-projekt.ps1" zum Anlegen neuer Projekte … Seite 22
Beispiel 2.18: Skriptdatei anlegen … Seite 22
Beispiel 2.19: Aufruf über eine Funktion im PowerShell-Profil … Seite 22
Beispiel 2.20: Inhalt der Profildatei (eine Zeile ergänzen) … Seite 23
Beispiel 2.21: Terminal neu starten und testen … Seite 23
Beispiel 3.1: Planungs-Prompt – erst planen, dann bauen … Seite 26
Beispiel 3.2: Projektnotiz PROJEKT.md als Gedächtnis … Seite 26
Beispiel 3.3: Prompt für einen kleinen, klaren Schritt … Seite 26
Beispiel 3.4: dupfinder.py, Schritt 1 – Dateien auflisten … Seite 27
Beispiel 3.5: Schritt 1 testen … Seite 27
Beispiel 3.6: Funktionierenden Stand sichern … Seite 28
Beispiel 3.7: Erweiterung mit klarer Eingrenzung … Seite 28
Beispiel 3.8: Änderungen der KI mit git diff kontrollieren … Seite 28
Beispiel 3.9: Vorlage für eine Fehleranfrage … Seite 29
Beispiel 3.10: Fehlerausgabe in eine Datei umleiten … Seite 29
Beispiel 3.11: Diagnose-Ausgaben mit logging … Seite 29
Beispiel 3.12: Projektstruktur kompakt für die KI ausgeben … Seite 30
Beispiel 3.13: Gezielte Auszüge statt ganzer Dateien … Seite 30
Beispiel 3.14: Lerntagebuch-Eintrag … Seite 31
Beispiel 4.1: Ein zu vager Prompt … Seite 33
Beispiel 4.2: Ein präziser Prompt … Seite 33
Beispiel 4.3: Rückfragen erzwingen … Seite 34
Beispiel 4.4: Mit Beispiel-Ein- und Ausgabe arbeiten … Seite 34
Beispiel 4.5: Selbstprüfung anfordern … Seite 34
Beispiel 4.6: Umgebungsinfos für die KI sammeln … Seite 35
Beispiel 4.7: Mehrere Dateien sauber übergeben … Seite 35
Beispiel 4.8: Traceback richtig lesen … Seite 35
Beispiel 4.9: Typische Shell-Fehler und ihre Bedeutung … Seite 35
Beispiel 4.10: Relevante Logzeilen herausfiltern … Seite 36
Beispiel 4.11: Review-Prompt … Seite 36
Beispiel 4.12: Statische Prüfwerkzeuge laufen lassen … Seite 37
Beispiel 4.13: Ein einfacher Test mit unittest (Standardbibliothek) … Seite 37
Beispiel 4.14: Tests ausführen … Seite 37

Beispiel 4.15: Tests vor der Änderung anfordern … Seite 37
Beispiel 5.1: Projekt anlegen … Seite 39
Beispiel 5.2: Prompt für das Gruppieren … Seite 39
Beispiel 5.3: dupfinder.py – vollständige Fassung … Seite 41
Beispiel 5.4: Testordner mit bekannten Duplikaten erzeugen … Seite 42
Beispiel 5.5: tests/test_dupfinder.py … Seite 43
Beispiel 5.5b: Zusätzlicher Test mit Verknüpfungen (Entwicklermodus oder Administratorrechte erforderlich) … Seite 43
Beispiel 5.6: Tests ausführen und committen … Seite 43
Beispiel 5.6b: Bedingte Ausführung ohne &&, kompatibel mit PowerShell 5.1 und 7 … Seite 44
Beispiel 5.7: dupfinder wie ein echtes Kommandozeilenprogramm installieren … Seite 44
Beispiel 5.7b: Aufruf über eine Funktion im PowerShell-Profil … Seite 44
Beispiel 5.7c: Terminal neu starten und testen … Seite 44
Beispiel 5.8: Prompt für die Aufgabenliste … Seite 45
Beispiel 5.9: index.html – Aufgabenliste (eine Datei) … Seite 46
Beispiel 5.10: Seite lokal ansehen … Seite 47
Beispiel 5.11: Zum Vergleich – der ungefilterte Aufruf (bewusst, mit Erklärung) … Seite 47
Beispiel 5.12: Flask installieren … Seite 47
Beispiel 5.13: app.py – minimaler Flask-Server mit JSON-Schnittstelle … Seite 48
Beispiel 5.14: Server starten und mit curl testen … Seite 48
Beispiel 5.15: Prompt für das Aufräumskript … Seite 49
Beispiel 5.16: aufraeumen.py … Seite 50
Beispiel 5.17: Test im Wegwerf-Ordner statt in den echten Downloads … Seite 51
Beispiel 5.18: Tägliche Aufgabe registrieren (zunächst als Trockenlauf) … Seite 52
Beispiel 5.19: Trockenlauf-Aufgabe gefahrlos testen … Seite 52
Beispiel 5.19b: Erst nach kontrollierter Sichtung auf echten Betrieb umstellen … Seite 52
Beispiel 5.19c: Aufgabe entfernen … Seite 52
Beispiel 6.1: Key aus der Umgebung lesen, nie im Code speichern … Seite 55
Beispiel 6.2: Nach Geheimnissen im Projekt suchen (nur im Git-Repository ausführen) … Seite 55
Beispiel 6.3: Trockenläufe und sichere Gewohnheiten … Seite 56
Beispiel 6.4: Windows Sandbox aktivieren (einmalig, Administrator-Terminal, nur Pro/Enterprise/Education) … Seite 57
Beispiel 6.5: Unsicher und sicher – Befehle aus Eingaben bauen … Seite 57
Beispiel 6.6: Pfade absichern … Seite 57
Beispiel 6.7: Parametrisierte SQL-Abfrage mit sqlite3 … Seite 57
Beispiel 6.8: Prüfroutine als kleines Skript … Seite 59
Beispiel 6.9: Ein Paket vor der Installation prüfen … Seite 60
Beispiel 6.10: Testdaten für eine Beispiel-CSV selbst erzeugen (keine echten Daten nötig) … Seite 61
Beispiel 7.1: Behauptungen schnell nachprüfen … Seite 64
Beispiel 7.2: Versionen feststellen … Seite 65
Beispiel 7.3: Versionsbewusster Prompt … Seite 65
Beispiel 7.4: Die richtige Umgebung prüfen … Seite 66
Beispiel 7.5: Virtuelle Umgebung sauber neu aufbauen … Seite 66
Beispiel 7.6: Prompt für Verständnis in Stufen … Seite 67
Beispiel 7.7: Verdächtige Fehlerbehandlung aufspüren … Seite 68
Beispiel 7.8: Zeichensatz und Zeilenenden prüfen und reparieren … Seite 68
Beispiel 8.1: Wochenplan für Stufe 2 … Seite 71
Beispiel 8.2: Der KI-Tutor-Prompt … Seite 71
Beispiel 8.3: Checkliste für jedes Projekt … Seite 74

Index

.gitignore – 21, 66
API-Key – 18, 19, 54
Aufgabenplanung – 52
Ausführungsrichtlinie – 16, 23
Branch – 71
Code-Review – 36, 73
Command Injection – 57
Commit – 20, 21, 25, 26, 27, 39, 43, 55
CRLF – 68
Datenschutz – 60, 61, 72
Debugging – 28
Exit-Code – 27
Flask – 47, 48, 58, 72
Git – 6, 9, 10, 13, 15, 16, 18, 20
git diff – 28, 58, 67, 68
git restore – 31, 67
Halluzination – 9, 64
Hash – 39
icacls – 19
jq – 44
Kontextfenster – 30
Lerntagebuch – 15, 31, 68
Linter – 37, 58
Lizenz – 60
localStorage – 45
Neovim – 18
Path Traversal – 57
PowerShell – 5, 6, 7, 13, 16, 19, 22, 29
PROJEKT.md – 26
Prompt – 25, 26, 33, 36, 37, 39, 45, 49
PSScriptAnalyzer – 37, 58
requirements.txt – 58
SHA-256 – 39
Slopsquatting – 59
SQL – 57
Standardbibliothek – 8, 30, 37, 60, 64, 67
Terminal-Agent – 9, 18, 26, 55
Traceback – 35
Trockenlauf – 48, 49, 52
Umgebungsvariable – 20, 54
unittest – 37, 58
UTF-8 – 68
venv – 15, 16, 66, 68
Vibe Coding – 5, 8, 20, 25, 54, 66, 73, 74
VS Code – 17, 68
Windows Sandbox – 56, 57
winget – 5, 13, 14
WSL – 18, 68
XSS – 47, 58

VIBE CODING UNTER WINDOWS

Eigene Software bauen – mit KI und PowerShell

Du kommst mit Windows und der Kommandozeile zurecht, hast aber noch nie programmiert? Dieses Handbuch zeigt dir, wie du mit KI-Assistenten eigene Werkzeuge baust: vom ersten Prompt bis zum getesteten Programm. Du richtest Windows Terminal, Python und Git ein, arbeitest in kleinen, sicheren Schritten und lernst, KI-Code zu prüfen, statt ihm blind zu vertrauen.

DAS ERWARTET DICH

- Setup unter Windows: Windows Terminal, PowerShell, winget, VS Code, Python, Git
- Windows-Besonderheiten: Ausführungsrichtlinie, UTF-8, Aufgabenplanung, WSL im Überblick
- Drei Projekte Schritt für Schritt: CLI-Tool, Web-Projekt, Automatisierungsskript
- Sicherheit und Qualität: Keys, Lizenzen, Datenschutz, Grenzen von KI-Code
- Übungen, Warn- und Tipp-Boxen, Glossar, Verzeichnis der Beispiele, Index

Für wen?

Für Einsteiger:innen mit Windows-Grundkenntnissen. Keine Vorkenntnisse im Programmieren nötig. Praxisnah, Schritt für Schritt, mit Übungen.

80 Seiten · Stand Oktober 2026 · Für Einsteiger:innen mit Windows-Grundkenntnissen

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.